

LỜI MỞ ĐẦU

Trong vận hành phần mềm, log là một trong những nguồn tín hiệu gần nhất với sự cố thực tế. Một dòng log có thể cho biết request nào thất bại, service nào phát sinh lỗi, mã truy vết nào liên quan, lỗi có lặp lại hay không và người vận hành nên bắt đầu điều tra từ đâu. Khi hệ thống chuyển từ một ứng dụng đơn lẻ sang nhiều service, log không còn là phần phụ trợ cho debug thủ công mà trở thành một dòng dữ liệu có tốc độ cao, có schema, có vòng đời lưu trữ và có yêu cầu phân quyền. Quan điểm này phù hợp với cách nhìn log như một abstraction thống nhất cho dữ liệu thời gian thực trong hệ thống phân tán [4, 1].

Đề tài LogRider được xây dựng trong bối cảnh đó. Hệ thống hướng tới việc tiếp nhận log ứng dụng, đếm qua message broker, chuẩn hóa payload, phân loại message, lưu trữ vào kho phân tích, cảnh báo với các lỗi nghiêm trọng và cung cấp giao diện dashboard thời gian gần thực. Phiên bản hiện tại không phải kiến trúc Rust/actor hoặc bản thiết kế lịch sử trong các tài liệu cũ. Mã nguồn thực tế là một ứng dụng Docker Compose gồm Redpanda/Pandaproxy, Benthos, ClickHouse, Redis, PostgreSQL, Bun web server, Node/Bun alert worker, Python classifier worker và Go Telegram bot. Báo cáo vì vậy ưu tiên đọc cấu hình, pipeline và source code hơn README.

Cách trình bày của báo cáo được tổ chức theo cấu trúc của một báo cáo mini-project kỹ thuật: mở đầu, tóm tắt, nội dung thực hiện, giới thiệu bài toán, phân tích yêu cầu, kiến trúc, thiết kế luồng xử lý, mô hình dữ liệu, phương pháp đánh giá, kết quả hiện trạng, hạn chế và hướng phát triển. Các phần benchmark trong phiên bản này được điền bằng kết quả chạy thật ngày 05/07/2026 trên môi trường Docker Compose cục bộ. Những chỉ số chưa được hệ thống instrument trực tiếp, ví dụ latency từng stage cho từng message hoặc độ chính xác classifier trên dataset có nhãn, được ghi rõ là chưa đo thay vì suy diễn.

TÓM TẮT NỘI DUNG VÀ ĐÓNG GÓP

Báo cáo trình bày LogRider như một hệ thống log monitoring theo kiến trúc event-driven. Dòng log đi vào Redpanda thông qua Pandaproxy ở topic logs-ingested. Pipeline unified.yaml chuẩn hóa payload, sinh Trace_ID nếu thiếu, phát sự kiện Ingested qua Redis và tách nhánh log ERROR/CRITICAL sang topic cảnh báo. Worker phân loại Python tiêu thụ logs-normalized, gắn tag cho message và chuyển log enriched sang topic logs-persist. Pipeline persist.yaml ghi batch vào ClickHouse bảng logrider.logs_enriched. Alert worker tiêu thụ alerts-ingested, dùng Redis để deduplicate theo application và message hash, sau đó phát sự kiện cảnh báo cho dashboard và tạo job gửi Telegram. Web server Bun cung cấp trang dashboard, alerts, metrics, REST API, authentication và WebSocket fan-out.

Đóng góp chính của báo cáo nằm ở việc tái cấu trúc mô tả hệ thống theo đúng implementation. Thứ nhất, báo cáo xác định lại kiến trúc runtime hiện tại và phân rã thành các vùng ingress, broker, processing, storage/state và presentation/notification. Thứ hai, báo cáo bổ sung phần phân tích yêu cầu theo dạng bảng chức năng và phi chức năng, tương tự các báo cáo mẫu có phần yêu cầu rõ ràng. Thứ ba, báo cáo mở rộng phần Nội dung và phương pháp với cơ sở lý thuyết, sequence diagram, thiết kế dữ liệu, topic catalog, Redis state, ClickHouse schema, API/RBAC và phương pháp đo lường. Thứ tư, báo cáo phân biệt rõ phần đã có trong mã nguồn với phần cần bổ sung bằng kết quả thực nghiệm, tránh biến báo cáo thành tài liệu quảng bá thiếu kiểm chứng.

Các phát hiện kỹ thuật quan trọng gồm: hệ thống đã có pipeline ingest-classify-persist và alert end-to-end ở mức prototype; ClickHouse được dùng phù hợp cho truy vấn log append-only; Redis được dùng rộng cho pub/sub, deduplication và trạng thái transient; PostgreSQL đang giữ vai trò user/RBAC cơ bản. Ngược lại, hệ thống còn các điểm cần xử lý trước khi triển khai thật: thiếu ingestion API có xác thực riêng, trạng thái xử lý chưa được lưu bền vững, live WebSocket có rủi ro RBAC cho non-admin, alert deduplication hiện gửi lần đầu và các threshold nên chưa tương thích nếu yêu cầu là đúng một notification trong một cửa sổ thời gian, ordering key ClickHouse cần được kiểm chứng theo workload truy vấn, và cấu hình vận hành còn phân tán.

NỘI DUNG THỰC HIỆN

Bảng 1 tóm tắt các nhóm công việc được trình bày trong báo cáo. Cấu trúc này học theo cách các báo cáo mẫu mở đầu bằng phần tổng kết các đầu việc lớn trước khi đi vào chương phân tích và triển khai.

Bảng 1: Tổng hợp nội dung thực hiện trong báo cáo

STT	Nhóm công việc	Nội dung chính
1	Khảo sát implementation	Đọc cấu trúc repository, Docker Compose, pipeline Benthos, schema ClickHouse/PostgreSQL, server, worker và integration để xác lập nguồn sự thật của hệ thống.
2	Phân tích yêu cầu	Chuyển chức năng hiện có thành nhóm yêu cầu chức năng, phi chức năng, phạm vi, giới hạn và các rủi ro vận hành cần đo.
3	Thiết kế kiến trúc	Tái dựng kiến trúc tổng quan, phân tách vùng ingress/broker/processing/storage/presentation và mô tả vai trò từng service.
4	Thiết kế luồng xử lý	Xây dựng sequence diagram cho luồng ingest–persist, cảnh báo, authentication/RBAC và Telegram notification.
5	Thiết kế dữ liệu	Mô tả schema log, bảng ClickHouse, bảng PostgreSQL, topic catalog, Redis key/state và mối quan hệ giữa dữ liệu bền vững với trạng thái transient.
6	Đánh giá hiện trạng	Tổng hợp phần đã hoàn thiện, phần có nhưng cần sửa, phần thiếu dữ kiện và các testcase cần chạy trước khi nghiệm thu.
7	Định hướng cải thiện	Đề xuất mở rộng benchmark, chuẩn hóa alert semantics, dashboard DLQ, migration và cấu hình vận hành.

Mục lục

Lời mở đầu	i
Tóm tắt nội dung và đóng góp	ii
Nội dung thực hiện	iii
I Giới thiệu	1
1 Bối cảnh đề tài	1
2 Mục tiêu và bài toán đặt ra	1
2.1 Yêu cầu chức năng	1
2.1.1 FR-01 – Tiếp nhận log qua một boundary thống nhất	2
2.1.2 FR-02 – Chuẩn hóa payload và bảo đảm định danh log	2
2.1.3 FR-03 – Phát trạng thái xử lý gần thời gian thực	2
2.1.4 FR-04 – Phân loại nội dung log bằng worker tách biệt	2
2.1.5 FR-05 – Lưu trữ log enriched vào kho phân tích	2
2.1.6 FR-06 – Cảnh báo log nghiêm trọng và giảm nhiễu	2
2.1.7 FR-07 – Truy vấn lịch sử và chỉ số sức khỏe	3
2.1.8 FR-08 – Xác thực và phân quyền theo application	3
2.1.9 FR-09 – Dashboard thời gian gần thực	3
2.1.10 FR-10 – Thông báo Telegram theo subscription	3
2.2 Yêu cầu phi chức năng	3
2.2.1 NFR-01 – Thông lượng ingest và backpressure	3
2.2.2 NFR-02 – Độ tin cậy ghi dữ liệu và DLQ	4
2.2.3 NFR-03 – Tính đúng và độ nhiễu của cảnh báo	4
2.2.4 NFR-04 – Bảo mật, phân quyền và cô lập dữ liệu	4
2.2.5 NFR-05 – Khả năng truy vấn và lifecycle dữ liệu	4
2.2.6 NFR-06 – Khả năng triển khai và cấu hình	4
2.2.7 NFR-07 – Observability của chính pipeline monitoring	4
3 Phạm vi và giới hạn hệ thống	5
4 Thông tin mã nguồn và dữ liệu đầu vào	5
II Nội dung và phương pháp	6
1 Cơ sở lý thuyết và định hướng thiết kế	6
1.1 Log như một dòng sự kiện	6
1.2 Message broker, routing và DLQ	6
1.3 Kho dữ liệu dạng cột cho log analytics	6
1.4 Alert deduplication và khả năng hành động	7
2 Kiến trúc tổng thể hệ thống	7
2.1 Catalog thành phần triển khai	8
2.1.1 Redpanda	9

	2.1.2	<i>Pandaproxy</i>	9
	2.1.3	<i>Benthos unified pipeline</i>	9
	2.1.4	<i>Classifier worker</i>	9
	2.1.5	<i>Benthos persist và tags pipelines</i>	9
	2.1.6	<i>Alert worker</i>	9
	2.1.7	<i>Redis, ClickHouse và PostgreSQL</i>	9
	2.1.8	<i>Bun web server và Go Telegram bot</i>	10
3		Catalog công nghệ và quyết định lựa chọn	10
	3.1	Redpanda và Pandaproxy	10
	3.1.1	Vai trò trong kiến trúc	10
	3.1.1.1	Lý do không dùng queue đơn giản hơn	10
	3.1.1.2	Quyết định sử dụng Pandaproxy	10
	3.2	Benthos cho routing, normalization và persistence	11
	3.2.1	Lý do lựa chọn Benthos	11
	3.2.1.1	Unified pipeline như content-based router	11
	3.2.1.2	Persist và tags pipeline như bulk-writer	11
	3.3	ClickHouse cho lưu trữ log analytics	11
	3.3.1	Đặc tính phù hợp với log	11
	3.3.1.1	Ordering key và workload thực tế	11
	3.3.1.2	Materialized view cho health metrics	12
	3.4	Redis cho runtime state, pub/sub và deduplication	12
	3.4.1	Vai trò đa nhiệm	12
	3.4.1.1	Trade-off về tính bền vững	12
	3.4.1.2	Lua và atomic update	12
	3.5	PostgreSQL cho user và RBAC	12
	3.5.1	Lý do tách identity khỏi log store	12
	3.5.1.1	Giới hạn của mô hình <i>allowed_apps</i> dạng text	13
	3.6	Bun web server và frontend tĩnh	13
	3.6.1	Quyết định gộp HTML, REST API và WebSocket	13
	3.6.1.1	Rủi ro khi hệ thống lớn hơn	13
	3.6.1.2	Server-side authorization cho stream	13
	3.7	Python classifier worker	13
	3.7.1	Lý do dùng Python và ONNX Runtime	13
	3.7.1.1	Chất lượng classifier cần được đo độc lập	13
	3.8	Node/Bun alert worker và Go Telegram bot	14
	3.8.1	Alert worker như policy engine	14
	3.8.1.1	Telegram bot như delivery adapter	14
	3.9	Docker Compose cho môi trường demo	14
	3.9.1	Lý do chọn Docker Compose	14
	3.9.1.1	Giới hạn production	14
	3.10	Tổng hợp trade-off công nghệ	14
4		Thiết kế luồng dữ liệu và sequence diagram	14
	4.1	Luồng ingest, classify và persist	15
	4.2	Luồng cảnh báo và deduplication	15
	4.3	Luồng authentication, API và RBAC	16
	4.4	Luồng liên kết Telegram	17
5		Thiết kế dữ liệu	18
	5.1	Schema log canonical	18

5.2	Catalog topic và trách nhiệm	19
5.3	ClickHouse và PostgreSQL	19
5.4	Redis runtime state	20
5.5	Sơ đồ dữ liệu rút gọn	20
6	Thiết kế API, giao diện và quyền truy cập	21
7	Phân tích quyết định thiết kế theo góc nhìn kỹ sư	22
7.1	Tại sao không ghi log trực tiếp vào storage	22
7.1.1	<i>Ràng buộc producer</i>	23
7.1.1.1	<i>Độ trễ và độ phụ thuộc</i>	23
7.1.1.2	<i>Khả năng thêm nhánh xử lý</i>	23
7.1.2	<i>Ràng buộc storage</i>	23
7.1.2.1	<i>Batching</i>	23
7.2	Tại sao cần Trace_ID ngay từ stage đầu	23
7.2.1	<i>Liên kết lifecycle</i>	23
7.2.1.1	<i>Một log, nhiều trạng thái</i>	23
7.2.1.2	<i>Giới hạn hiện tại</i>	23
7.3	Tại sao tách alert khỏi classifier	24
7.3.1	<i>Độ ưu tiên xử lý</i>	24
7.3.1.1	<i>Alert không nên chờ ML</i>	24
7.3.1.2	<i>Noise và dedup</i>	24
7.4	Tại sao dashboard cần cả lịch sử và stream	24
7.4.1	<i>Hydration ban đầu</i>	24
7.4.1.1	<i>Historical API</i>	24
7.4.2	<i>Delta thời gian thực</i>	24
7.4.2.1	<i>WebSocket fan-out</i>	24
8	Phương pháp kiểm thử và đo lường	25
9	Phương pháp đối chiếu với tài liệu mẫu	27
III Kết quả thực hiện và đánh giá		28
1	Môi trường triển khai	28
2	Kết quả hiện thực theo mã nguồn	28
3	Đánh giá luồng xử lý	29
4	Đánh giá bảo mật và phân quyền	30
5	Đánh giá dữ liệu và truy vấn	31
6	Kịch bản thử nghiệm hiện có trong mã nguồn	31
7	Bảng kết quả thực nghiệm	32
8	Đánh giá tổng hợp	32
IV Kết luận		33
1	Tổng kết	33
2	Kết quả đạt được	33
3	Hạn chế	33
4	Hướng phát triển	33
Tài liệu tham khảo		35

Danh sách hình vẽ

1	Kiến trúc runtime của LogRider theo luồng kết nối chính	8
2	Sequence diagram luồng tiếp nhận, phân loại và lưu trữ log	15
3	Sequence diagram luồng cảnh báo, deduplication và notification	16
4	Sequence diagram theo phong cách PlantUML cho authentication và truy vấn log có RBAC	17
5	Sequence diagram theo phong cách PlantUML cho liên kết Telegram và gửi thông báo	18
6	Mô hình dữ liệu rút gọn của log, alert và RBAC	21
7	Khung đồ thị độ trễ pipeline theo từng stage	26
8	Khung đồ thị throughput ingest theo thời gian	26
9	Khung đồ thị kết quả xử lý end-to-end	26
10	Khung đồ thị đánh giá hành vi deduplication của alert	30

Danh sách bảng

1	Tổng hợp nội dung thực hiện trong báo cáo	iii
2	Schema log canonical và ý nghĩa vận hành	18
3	Catalog topic trong pipeline LogRider	19
4	Nhóm dữ liệu bền vững trong ClickHouse/PostgreSQL	20
5	Nhóm trạng thái Redis trong LogRider	20
6	Nhóm giao diện và API serving	21
7	Ma trận RBAC dự kiến cho dashboard và stream	22
8	Ma trận kiểm thử và đánh giá đề xuất	25
9	Các service chính trong môi trường Docker Compose	28
10	Mức độ hoàn thiện chức năng theo mã nguồn	28
11	Ma trận rủi ro kỹ thuật và hướng giảm thiểu	30
12	Khung workload truy vấn cần đo trên ClickHouse	31
13	Script kiểm thử và vận hành hiện có	31
14	Bảng kết quả benchmark đã đo	32

I. GIỚI THIỆU

Chương này xác định bối cảnh, mục tiêu, yêu cầu và giới hạn của đề tài. Thay vì bắt đầu ngay bằng danh sách yêu cầu, phần giới thiệu trình bày trước vấn đề vận hành khiến một hệ thống như LogRider cần tồn tại, sau đó mới chuyển sang các mục tiêu kỹ thuật và phạm vi đánh giá. Cách tổ chức này giúp người đọc hiểu vì sao từng quyết định kiến trúc ở các chương sau là hệ quả của yêu cầu thực tế chứ không phải lựa chọn công nghệ rời rạc.

1 Bối cảnh đề tài

Trong các nền tảng phần mềm có nhiều service, sự cố thường xuất hiện dưới dạng chuỗi sự kiện thay vì một lỗi cô lập. Một timeout tại API có thể bắt nguồn từ database, retry ở worker có thể làm tăng tải broker, và cùng một lỗi có thể sinh hàng trăm bản ghi log trong vài giây. Nếu thiếu pipeline thu thập tập trung, kỹ sư vận hành phải đọc log theo từng container, đối chiếu timestamp thủ công và tự quyết định lỗi nào đáng báo động. Cách làm này không mở rộng tốt khi số lượng service, môi trường và người dùng tăng lên.

Một hệ thống log monitoring cần giải quyết đồng thời ba nhóm bài toán. Nhóm thứ nhất là dữ liệu: log cần được chuẩn hóa để có thể lọc theo application, level, timestamp và trace identifier. Nhóm thứ hai là xử lý: dữ liệu cần được đệm, phân loại, lưu trữ và cảnh báo mà không khóa producer vào tốc độ consumer. Nhóm thứ ba là vận hành: người dùng phải xem được dashboard, nhận cảnh báo đúng mức độ, truy vấn lịch sử và bị giới hạn theo quyền. Các bài toán này liên quan trực tiếp tới các pattern message channel, content-based router, dead-letter channel và publish-subscribe channel trong tích hợp hệ thống [2, 3].

LogRider là một prototype nhằm hiện thực hóa lát cắt này. Hệ thống không cố xây dựng một full observability platform có metric, trace, service map và incident management đầy đủ. Trọng tâm của phiên bản hiện tại là log pipeline: ingest, normalize, classify, persist, alert và dashboard. Vì vậy, báo cáo đánh giá hệ thống trong phạm vi log monitoring, không mở rộng claim sang APM hoặc SIEM production-grade.

2 Mục tiêu và bài toán đặt ra

Mục tiêu tổng quát của mini-project là xây dựng một hệ thống log collection và application error monitoring có thể chạy bằng Docker Compose. Hệ thống cần tiếp nhận log với định dạng thống nhất, không làm producer phụ thuộc trực tiếp vào storage, lưu log vào kho phân tích phù hợp với truy vấn theo thời gian, phát hiện lỗi nghiêm trọng, giảm nhiễu cảnh báo khi cùng lỗi lặp lại và cung cấp dashboard có phân quyền. Từ mục tiêu này, bài toán kỹ thuật cốt lõi là thiết kế một luồng end-to-end trong đó mỗi stage có trách nhiệm rõ ràng và có thể kiểm thử độc lập.

2.1 Yêu cầu chức năng

Các yêu cầu chức năng của LogRider được trình bày dưới dạng các tiểu mục thay vì bảng để làm rõ động cơ thiết kế phía sau từng capability. Với một hệ thống log monitoring, việc mô tả yêu cầu chỉ bằng một dòng thường không đủ vì mỗi chức năng đều kéo theo một quyết định kiến trúc: ingest cần broker, cảnh báo cần runtime state, dashboard cần RBAC, còn persistence cần storage tối ưu cho truy vấn theo thời gian. Cách trình bày này cũng giúp người đọc thấy một quyết định kỹ thuật xuất hiện từ nhu cầu vận hành nào, thay vì chỉ thấy danh sách tính năng rời rạc.

Trong phần này, mỗi yêu cầu được viết theo ba lớp ý nghĩa: năng lực hệ thống phải cung cấp, lý do năng lực đó quan trọng đối với vận hành log pipeline, và rủi ro nếu năng lực đó chỉ được triển khai ở mức demo. Đây là cách viết gần với tài liệu kỹ thuật của kỹ sư hơn là checklist sản phẩm, vì nó buộc từng yêu cầu phải gắn với một trade-off cụ thể.

2.1.1 FR-01 – Tiếp nhận log qua một boundary thống nhất

Hệ thống cần có một điểm tiếp nhận log duy nhất để producer không phải biết các thành phần xử lý phía sau. Trong implementation hiện tại, producer gửi Kafka REST payload vào Pandaproxy và topic logs-ingested. Quyết định dùng một topic ingress chung giúp toàn bộ pipeline có cùng một điểm bắt đầu, dễ kiểm thử và dễ đo lag. Tuy nhiên, đây mới là boundary thuận tiện cho demo, chưa phải ingestion API production. Một ingestion API đầy đủ cần xác thực API key, kiểm tra schema, rate limit theo application, ghi audit và trả lỗi có semantics ổn định. Nếu thiếu lớp này, producer bên ngoài phải biết trực tiếp Pandaproxy và hệ thống khó áp dụng policy theo tenant/application.

2.1.2 FR-02 – Chuẩn hóa payload và bảo đảm định danh log

Pipeline pipelines/unified.yaml xử lý trường hợp payload có lớp value, giữ nguyên payload nếu không có value, và sinh Trace_ID bằng UUID khi thiếu. Quyết định đặt bước này ở đầu pipeline là hợp lý vì mọi nhánh sau đó, gồm classifier, persistence, alert và dashboard, đều cần một khóa định danh ổn định để cập nhật cùng một dòng log qua nhiều trạng thái. Nếu trace id chỉ được sinh ở worker sau, các event như Ingested và Persisted sẽ khó nối với nhau ở UI.

2.1.3 FR-03 – Phát trạng thái xử lý gần thời gian thực

Dashboard cần phản ánh vòng đời của log, không chỉ hiển thị bản ghi đã lưu. Vì vậy pipeline và worker phát các trạng thái như Ingested, classification/tag event và Persisted qua Redis pub/sub để web server gửi tiếp cho browser bằng WebSocket. Quyết định dùng Redis pub/sub phù hợp với prototype vì đây là trạng thái transient, có độ trễ thấp và không cần lưu bền vững. Trade-off là trạng thái này mất nếu client không online hoặc Redis restart. Nếu yêu cầu audit vòng đời từng log, trạng thái xử lý phải được ghi thêm vào storage bền vững.

2.1.4 FR-04 – Phân loại nội dung log bằng worker tách biệt

Worker Python tiêu thụ logs-normalized, batch message, dùng mô hình text classification qua Hugging Face/ONNX Runtime, gắn Tags và produce bản ghi enriched sang logs-persist. Việc tách classifier khỏi web server là quyết định kỹ thuật quan trọng: thư viện machine learning nặng, có latency biến động và có thể cần tài nguyên riêng. Nếu nhúng classifier vào web server, lỗi model hoặc batch inference chậm có thể kéo theo API/dashboard. Worker riêng cũng tạo điều kiện scale độc lập khi khối lượng log tăng.

2.1.5 FR-05 – Lưu trữ log enriched vào kho phân tích

Pipeline pipelines/persist.yaml tiêu thụ logs-persist, ghi batch vào ClickHouse bằng logrider.logs_enriched theo định dạng JSONEachRow, đồng thời fallback sang dlq-clickhouse khi lỗi. Quyết định dùng ClickHouse phản ánh tính chất append-only và truy vấn theo thời gian của log. Quyết định dùng batch insert phản ánh đặc tính của ClickHouse: nhiều insert nhỏ thường kém hiệu quả hơn một batch lớn. Hệ thống cần tiếp tục làm rõ thời điểm phát Persisted; về mặt semantics, trạng thái này nên được phát sau khi insert thành công chứ không chỉ sau khi message đi vào pipeline persist.

2.1.6 FR-06 – Cảnh báo log nghiêm trọng và giảm nhiễu

Log có Log_Level là ERROR hoặc CRITICAL được route sang alerts-ingested. Alert worker thực hiện deduplication theo application và message hash bằng Redis, sau đó phát alert cho dashboard và Telegram. Lý do tách alert path từ unified.yaml là lỗi nghiêm trọng không nên phải chờ classifier. Severity thô đủ để kích hoạt cảnh báo ban đầu, còn tags có thể bổ sung ngữ cảnh sau. Phần cần quyết định rõ là semantics của deduplication: hệ

thống hiện gửi lần đầu và gửi lại tại các mốc threshold, trong khi một số yêu cầu vận hành có thể muốn đúng một notification trong một cửa sổ thời gian.

2.1.7 FR-07 – Truy vấn lịch sử và chỉ số sức khỏe

Bun web server truy vấn ClickHouse để trả API log gần đây, bảng xếp hạng lỗi và dữ liệu chart. ClickHouse có materialized view tổng hợp `error_count` và `total_count` theo giờ/application. Lý do tạo aggregate là dashboard không nên scan toàn bộ bảng log cho mọi refresh nếu chỉ cần error rate theo giờ. Quyết định này giúp giảm chi phí truy vấn, nhưng cần kiểm chứng bằng workload thật để biết materialized view hiện có đủ cho các filter phổ biến hay chưa.

2.1.8 FR-08 – Xác thực và phân quyền theo application

PostgreSQL lưu `users`, `password_hash`, `role` và `allowed_apps`. Admin xem toàn bộ, engineer bị giới hạn theo application. Đây là phân quyền tối thiểu nhưng cần thiết vì log có thể chứa thông tin nhạy cảm giữa các nhóm ứng dụng khác nhau. Với historical API, filter có thể áp dụng ở truy vấn ClickHouse. Với WebSocket, filter phải áp dụng trước khi server gửi từng event cho client; nếu non-admin subscribe global stream thì UI filter không đủ bảo mật.

2.1.9 FR-09 – Dashboard thời gian gần thực

Web server nâng cấp WebSocket, subscribe Redis pub/sub và forward event tới browser. Dashboard cần ghép event live với dữ liệu lịch sử, cập nhật tags, trạng thái và cảnh báo mà không refresh toàn trang. Quyết định dùng WebSocket thay vì polling giúp giảm độ trễ và giảm request lặp. Tuy nhiên, WebSocket làm authorization khó hơn REST vì quyền phải được kiểm tra khi mở kết nối và khi forward từng event.

2.1.10 FR-10 – Thông báo Telegram theo subscription

Bot Go hỗ trợ `/link`, `/subscribe`, `/unsubscribe`, `/status` và tiêu thụ queue `telegram_outbound`. Lý do dùng token liên kết là người dùng không cần nhập credential vào Telegram; web server sinh token TTL ngắn, bot xác minh token qua Redis và gắn chat id với user/app scope. Quyết định này đơn giản và an toàn hơn so với truyền password qua bot, nhưng cần bổ sung cơ chế revoke, retry gửi message và lịch sử delivery nếu triển khai vận hành thật.

2.2 Yêu cầu phi chức năng

Yêu cầu phi chức năng là nơi phản ánh chất lượng kiến trúc. Một hệ thống log monitoring có thể chạy đúng trong demo nhưng vẫn chưa đạt yêu cầu vận hành nếu không chứng minh được thông lượng, độ tin cậy, phân quyền, khả năng quan sát nội bộ và khả năng phục hồi khi dependency lỗi. Các yêu cầu dưới đây được viết theo hướng có thể kiểm chứng bằng thí nghiệm hoặc code review.

Đối với LogRider, phần phi chức năng đặc biệt quan trọng vì hệ thống nằm trên đường vận hành sự cố. Nếu pipeline mất log, gửi sai cảnh báo hoặc lộ log giữa các application, lỗi không chỉ là lỗi phần mềm mà còn ảnh hưởng trực tiếp đến quá trình điều tra incident. Vì vậy các yêu cầu phi chức năng được mô tả bằng ngôn ngữ có thể kiểm chứng, kèm phần số liệu cần bổ sung nếu mã nguồn hiện tại chưa đủ chứng minh.

2.2.1 NFR-01 – Thông lượng ingest và backpressure

Hệ thống cần tiếp nhận burst log mà không làm producer bị phụ thuộc trực tiếp vào tốc độ ClickHouse hoặc classifier. Redpanda đóng vai trò buffer, còn Benthos persist dùng memory buffer và batch policy. Quyết định này hợp lý vì throughput của pipeline không đồng đều: classifier có thể chậm hơn broker, ClickHouse có thể tạm thời

lỗi, còn dashboard chỉ cần nhận subset event để hiển thị. Benchmark cục bộ trên máy Linux 20 logical CPU, RAM khoảng 15.4 GiB, Docker 29.5.3 và k6 2.0.0 cho thấy endpoint ingest có thể nhận burst k6 khoảng 250 request/giây với p95 HTTP acknowledgement 8.67 ms, nhưng drain sang ClickHouse trong kịch bản demo 500 log mất 31.7 giây. Vì vậy báo cáo chỉ claim khả năng demo end-to-end, chưa claim năng lực production throughput.

2.2.2 NFR-02 – Độ tin cậy ghi dữ liệu và DLQ

Khi ClickHouse lỗi, pipeline không nên làm mất log một cách im lặng. `persist.yaml` đã có fallback sang `dlq-clickhouse`, còn `tags.yaml` fallback sang `dlq-logs`. Đây là quyết định đúng vì lỗi storage là lỗi thường gặp trong pipeline dữ liệu. Tuy nhiên, DLQ chỉ có giá trị nếu có dashboard, alert và replay procedure. Vì vậy hệ thống cần bổ sung số lượng message trong DLQ, thời điểm lỗi, nguyên nhân lỗi, script replay và quy định khi nào operator được xóa DLQ.

2.2.3 NFR-03 – Tính đúng và độ nhiều của cảnh báo

Cảnh báo phải cân bằng giữa không bỏ sót lỗi nghiêm trọng và không spam người vận hành. Alert worker dùng Redis/Lua state để cập nhật deduplication nguyên tử, giúp tránh race condition khi nhiều event cùng lỗi đến gần nhau. Quyết định này phù hợp với bài toán hot-path runtime state. Tuy nhiên, chính sách gửi lần đầu và gửi lại tại threshold cần được viết thành contract rõ ràng. Nếu yêu cầu là “100 lần trong 1 phút chỉ gửi đúng một thông báo”, policy hiện tại cần thay đổi từ threshold reminder sang suppress-window notification.

2.2.4 NFR-04 – Bảo mật, phân quyền và cô lập dữ liệu

Log có thể chứa dữ liệu nhạy cảm như endpoint, user id, stack trace hoặc thông tin nội bộ. Vì vậy RBAC không chỉ là tính năng UI mà là yêu cầu bảo mật. PostgreSQL cung cấp source of truth cho user/role/allowed apps. Historical API có thể lọc query theo application, nhưng WebSocket cần lọc event trước khi forward. Quyết định quan trọng ở đây là luôn thực thi authorization ở server, không tin tưởng frontend. Hệ thống hiện cần sửa live stream để non-admin không nhận global log topic.

2.2.5 NFR-05 – Khả năng truy vấn và lifecycle dữ liệu

ClickHouse dùng MergeTree, native type cho level/timestamp/UUID và TTL 7 ngày. Đây là lựa chọn hợp lý cho prototype vì log không cần lưu vô hạn trong môi trường demo. Tuy nhiên, ordering key (Timestamp, Trace_ID) ưu tiên scan theo thời gian, chưa chắc tối ưu nếu truy vấn phổ biến là application + level + time range. Cần đo workload dashboard và query thực tế trước khi đổi ordering key hoặc thêm projection. Quyết định tối ưu storage nên dựa trên query profile, không dựa trên cảm giác.

2.2.6 NFR-06 – Khả năng triển khai và cấu hình

Docker Compose giúp người đánh giá dựng stack nhanh và tái lập môi trường. Đây là lựa chọn phù hợp với mini-project có nhiều dependency. Dù vậy, Compose không chứng minh high availability, autoscaling, rolling deploy hoặc secret management. Hệ thống còn cần chuẩn hóa biến môi trường, topic name, port, credential demo, image tag và script setup để giảm sai khác giữa local, CI và môi trường triển khai.

2.2.7 NFR-07 – Observability của chính pipeline monitoring

Một hệ thống monitoring cũng cần được monitor. LogRider hiện có dashboard ứng dụng và metrics page, nhưng cần thêm signal nội bộ: consumer lag, insert failure, DLQ size, classifier batch latency, Redis pub/sub health, Telegram

delivery failure và WebSocket client count. Đây là yêu cầu quan trọng vì nếu pipeline monitoring bị lỗi, người vận hành cần biết lỗi nằm ở producer, broker, worker, storage hay UI.

3 Phạm vi và giới hạn hệ thống

Phạm vi của báo cáo là thư mục `logrider/` trong gói mã nguồn: Docker Compose, Redpanda/Pandaproxy, Benthos pipeline, ClickHouse/PostgreSQL initialization, Redis state, Bun server, alert worker, classifier worker, Telegram integration và scripts vận hành. Các file đặc tả lịch sử hoặc README cũ chỉ được dùng làm ngữ cảnh phụ, không được xem là nguồn sự thật nếu khác code hiện tại.

Giới hạn quan trọng nhất là benchmark mới được chạy trên Docker Compose cục bộ, chưa phải môi trường production nhiều node. Vì vậy, các biểu đồ hiệu năng trong báo cáo được hiểu là số liệu prototype trên một máy cụ thể. Giới hạn thứ hai là hệ thống hiện chưa có Kubernetes, autoscaling, secret management, centralized logging cho chính hệ thống, backup/restore và CI/CD production-grade. Giới hạn thứ ba là classifier có implementation và benchmark throughput, nhưng chưa có thông tin về tập huấn luyện, tập kiểm thử và metric đánh giá, nên báo cáo không kết luận chất lượng mô hình.

4 Thông tin mã nguồn và dữ liệu đầu vào

Mã nguồn được cung cấp dưới dạng một file Repomix gồm cấu trúc thư mục và nội dung các file quan trọng. Template báo cáo ban đầu đến từ `layout.docx`. Phong cách viết và cách tổ chức nội dung được tham chiếu từ các PDF báo cáo mẫu và từ file `main-110626.tex` trong `content.zip`, đặc biệt là cách trình bày yêu cầu chức năng, yêu cầu phi chức năng, sequence diagram và bảng đánh giá. Các thông tin không có trong nguồn đầu vào được giữ dưới dạng placeholder, ví dụ `<PLACEHOLDER: {student name}>`, `<PLACEHOLDER: {student email}>`, `<PLACEHOLDER: {mentor name}>`, `<PLACEHOLDER: {organization unit}>`, `<PLACEHOLDER: {public repository link if available}>`.

II. NỘI DUNG VÀ PHƯƠNG PHÁP

Chương này trình bày phần lõi kỹ thuật của báo cáo: cơ sở lý thuyết, kiến trúc runtime, tech stack, luồng dữ liệu, mô hình lưu trữ, API và phương pháp kiểm thử. Trọng tâm không chỉ là mô tả thành phần nào tồn tại trong mã nguồn, mà còn phân tích vì sao thành phần đó được đặt ở đúng vị trí trong pipeline và trade-off nào phát sinh từ quyết định đó.

1 Cơ sở lý thuyết và định hướng thiết kế

Phần cơ sở lý thuyết đặt nền cho các quyết định thiết kế của LogRider. Mỗi khái niệm được chọn vì nó có liên hệ trực tiếp với implementation: log được xem là event stream, broker đảm nhiệm decoupling, DLQ bảo vệ đường ghi, column store phục vụ truy vấn analytics và deduplication làm giảm nhiễu cảnh báo.

1.1 Log như một dòng sự kiện

Trong hệ thống dữ liệu hiện đại, log có thể được hiểu như một chuỗi sự kiện append-only có thứ tự tương đối theo thời gian. Mỗi event không nhất thiết là trạng thái cuối cùng của hệ thống, nhưng có thể được dùng để tái dựng, tổng hợp, phát hiện bất thường hoặc kích hoạt workflow. Cách nhìn này giúp giải thích vì sao broker và consumer group phù hợp với bài toán log: producer chỉ cần append event, còn các consumer phía sau có thể xử lý theo nhánh khác nhau như persistence, classification và alert [4, 1].

Với LogRider, log không được ghi trực tiếp vào ClickHouse ngay tại producer. Thay vào đó, producer đẩy vào logs-ingested; pipeline sau đó normalize, classifier enrich, persist ghi vào ClickHouse và alert worker xử lý lỗi nghiêm trọng. Kiến trúc này làm tăng số thành phần nhưng giảm coupling và cho phép từng nhánh xử lý có tốc độ, lỗi và chính sách retry riêng. Đây là trade-off phổ biến trong hệ thống phân tán: đơn giản hóa producer bằng cách đưa độ phức tạp vào pipeline xử lý và observability nội bộ [2, 3].

1.2 Message broker, routing và DLQ

Message broker đóng vai trò đệm và phân phối. Trong LogRider, Redpanda được dùng như broker tương thích Kafka, còn Benthos đóng vai trò xử lý pipeline và router. Các pattern liên quan gồm content-based router cho nhánh ERROR/CRITICAL, publish-subscribe channel cho Redis WebSocket event, và dead-letter channel cho lỗi insert ClickHouse. Những pattern này không chỉ là thuật ngữ kiến trúc mà là cơ sở để thiết kế testcase: với mỗi nhánh route cần có một input mẫu, một output mong đợi và một cách kiểm tra lỗi [2].

DLQ đặc biệt quan trọng vì hệ thống log thường chấp nhận dữ liệu lớn và không đồng nhất. Khi ClickHouse tạm thời không ghi được, việc đẩy bản ghi sang dlq-clickhouse giúp tránh mất dữ liệu ngay lập tức. Tuy nhiên, DLQ chỉ hữu ích nếu có dashboard, alert và cơ chế replay. Hiện mã nguồn có fallback topic, nhưng báo cáo chưa thấy quy trình replay hoặc thống kê DLQ, nên phần này được đánh giá là có nền tảng nhưng chưa hoàn chỉnh.

1.3 Kho dữ liệu dạng cột cho log analytics

Log monitoring thường có pattern truy vấn theo thời gian, application, level, tag và trace identifier. Kho dạng cột phù hợp vì nhiều truy vấn chỉ đọc một số cột trong lượng dữ liệu lớn. ClickHouse bằng logs_enriched dùng MergeTree, TTL 7 ngày và ordering key (Timestamp, Trace_ID). Thiết kế này hợp lý cho timeline append-only, nhưng chưa chắc tối ưu cho dashboard thường lọc theo Application_Name hoặc Log_Level. Vì vậy, việc tối ưu schema cần dựa trên workload và kết quả đo, tương tự nguyên tắc thiết kế storage luôn phụ thuộc vào truy vấn thực tế [9, 13].

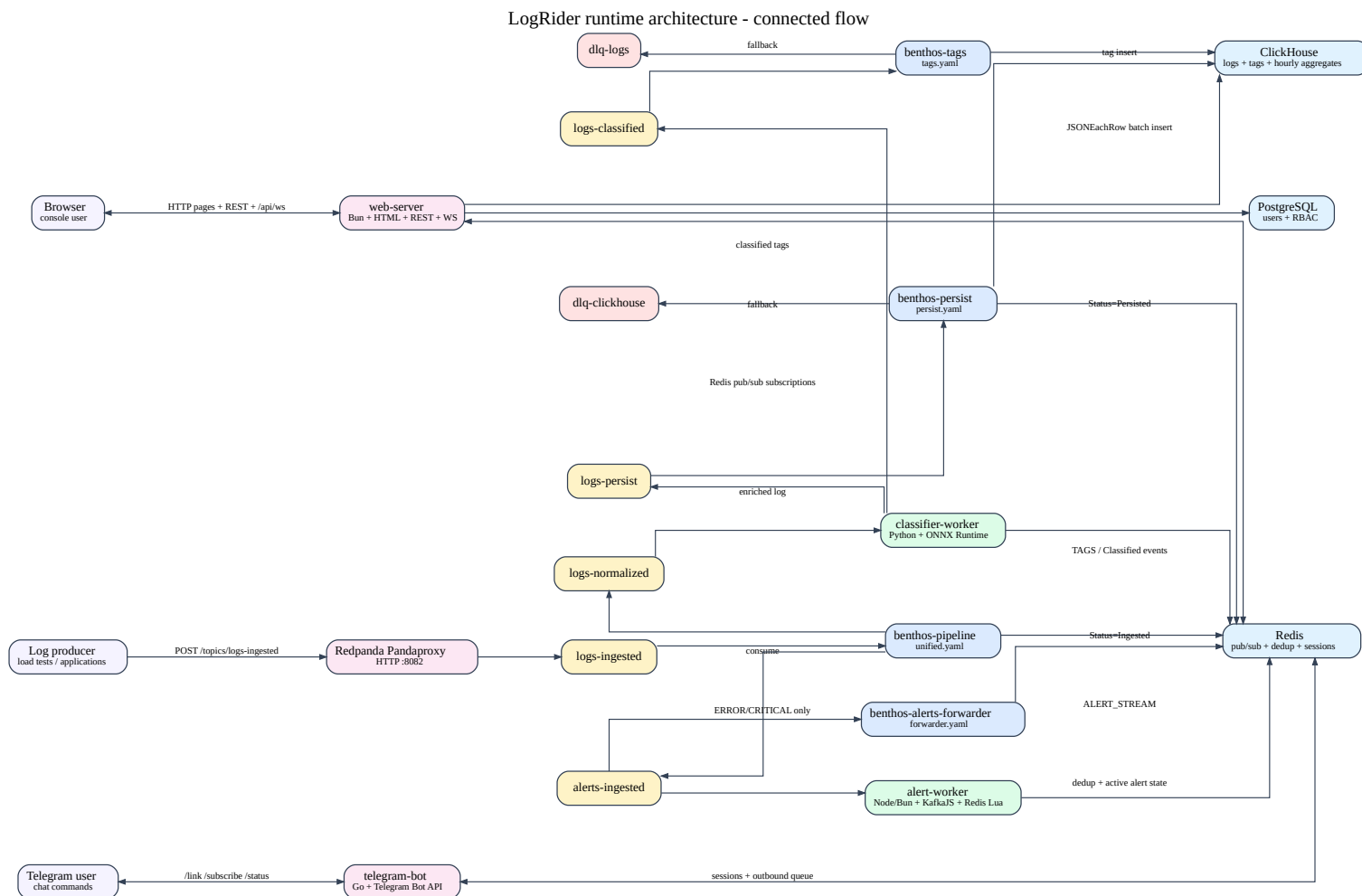
1.4 Alert deduplication và khả năng hành động

Alert không nên chỉ phản ánh mọi lỗi thô. Nếu cùng một lỗi xuất hiện 500 lần trong một phút, gửi 500 thông báo sẽ làm người trực mất khả năng phân biệt tín hiệu quan trọng. Alert deduplication vì vậy cần gom lỗi giống nhau theo một key ổn định, chẳng hạn application và hash của message, rồi quyết định khi nào gửi notification. SRE nhấn mạnh rằng monitoring và alerting phải hướng tới hành động được, giảm noise và ưu tiên những tín hiệu cần can thiệp [5, 7].

Trong LogRider, alert worker có Redis-backed state và dùng Lua để cập nhật đếm/dedup. Cách làm này phù hợp với yêu cầu cập nhật nguyên tử trong một runtime store nhanh. Tuy nhiên, semantics sản phẩm cần thống nhất. Nếu yêu cầu là gửi lần đầu và nhắc lại tại các ngưỡng 10, 50, 100, implementation hiện tại có cơ sở. Nếu yêu cầu là đúng một notification cho 100 lỗi giống nhau trong một phút, implementation cần thay đổi theo window và testcase phải assert số notification bằng 1.

2 Kiến trúc tổng thể hệ thống

Kiến trúc tổng thể được chia thành năm vùng: actors, ingress/broker, processing, storage/state và presentation/notification. Cách chia này tương tự các sơ đồ trong báo cáo mẫu: mỗi vùng có nền màu riêng, các mũi tên ngắn, nhãn ngắn và không đặt text vượt ra ngoài box. Hình 1 trình bày kiến trúc runtime hiện tại của hệ thống.



Hình 1 được tổ chức theo bốn lane xử lý thay vì chỉ gom các khối thành từng cụm. Lane đầu tiên mô tả hot path từ producer tới Pandaproxy, Redpanda và pipeline chuẩn hóa. Lane thứ hai tách riêng nhánh classifier và persistence để làm rõ tại sao log được đưa qua topic trung gian trước khi ghi vào ClickHouse. Lane thứ ba thể hiện nhánh cảnh báo nghiêm trọng, trong đó alert worker cập nhật dedup state trước khi đẩy notification. Lane cuối cùng đặt web server, RBAC, truy vấn lịch sử và stream runtime vào cùng một vùng để cho thấy dashboard không đọc trực tiếp dữ liệu thô mà đi qua server-side authorization.

2.1 Catalog thành phần triển khai

2.1.1 Redpanda

Redpanda là broker tương thích Kafka, lưu các topic pipeline và DLQ. Thành phần này làm nhiệm vụ hấp thụ burst log, tách tốc độ producer khỏi tốc độ consumer và cho phép nhiều nhánh xử lý đọc cùng một dòng sự kiện. Khi vận hành, Redpanda cần được theo dõi ở các chỉ số topic retention, partition count, consumer group lag và disk usage. Script `scripts/setup-topics.sh` hiện có dấu hiệu tạo logs-ingested lặp với cấu hình partition khác nhau, nên cần dọn để tránh sai lệch giữa tài liệu và trạng thái cluster.

2.1.2 Pandaproxy

Pandaproxy là REST proxy của Redpanda, nhận Kafka JSON payload từ các producer demo. Lợi ích của Pandaproxy là giảm chi phí tích hợp ban đầu: script shell hoặc k6 có thể gửi HTTP request thay vì cấu hình Kafka client. Giới hạn là Pandaproxy không thay thế một ingestion gateway có xác thực và schema validation. Trong Compose hiện tại, Pandaproxy không được publish ra host như các script cũ giả định, vì vậy các script gọi `localhost:8082` cần được chạy trong network container hoặc cấu hình lại port.

2.1.3 Benthos unified pipeline

`pipelines/unified.yaml` là pipeline đầu tiên sau broker. Nó unwrap payload, sinh `Trace_ID`, phát trạng thái `Ingested`, gửi log bình thường sang classifier và route log nghiêm trọng sang alert topic. Quyết định đưa nhiều thao tác nhẹ vào unified stage giúp chuẩn hóa contract cho các nhánh sau. Rủi ro là pipeline YAML trở thành logic quan trọng cần test như code, không nên chỉ xem là cấu hình phụ.

2.1.4 Classifier worker

`workers/classifier/main.py` tiêu thụ logs-normalized, chạy phân loại nội dung log và produce tag event. Worker này là điểm đưa machine learning vào pipeline. Benchmark 100 log cho thấy classifier path tạo đủ tag với throughput 13.88 log/giây end-to-end, nhưng vẫn cần đo thêm theo message dài, batch size khác nhau và dataset có nhãn. Vì vậy classifier nên được mô tả là enrichment đã chạy được, chưa phải bộ phân loại đã chứng minh chất lượng.

2.1.5 Benthos persist và tags pipelines

`pipelines/persist.yaml` ghi logs-persist vào ClickHouse và phát `Persisted`. `pipelines/tags.yaml` ghi tag records vào `logrider.log_tags`. Lý do tách pipeline là log enriched và tag records có thể có schema và failure mode khác nhau. Khi vận hành, cần thống nhất xem tags là một phần của logs_enriched.Tags hay một bảng phụ join theo Trace_ID; nếu cả hai cùng tồn tại, dashboard và query phải có quy tắc ưu tiên rõ ràng.

2.1.6 Alert worker

`workers/alert/index.js` thực hiện deduplication lỗi bằng Redis, phát alert runtime và tạo Telegram outbound job. Thành phần này cần được thiết kế đặc biệt cẩn thận vì nó trực tiếp ảnh hưởng tới độ nhiễu vận hành. Một alert worker tốt không chỉ gửi thông báo mà còn cần giải thích vì sao thông báo được gửi: lỗi mới, đạt threshold, hết TTL hay thay đổi trạng thái.

2.1.7 Redis, ClickHouse và PostgreSQL

Redis là runtime state cho pub/sub, alert dedup, session/link token và queue ngắn hạn. ClickHouse là storage phân tích cho log enriched, tags và aggregate theo giờ. PostgreSQL là storage giao dịch nhỏ cho user/RBAC. Việc dùng

ba loại storage khác nhau là có lý do: mỗi loại dữ liệu có pattern truy cập khác nhau. Trộn tất cả vào một database sẽ đơn giản hơn về triển khai nhưng làm giảm tính phù hợp của từng workload.

2.1.8 Bun web server và Go Telegram bot

Bun web server phục vụ HTML, REST API, authentication, ClickHouse query và WebSocket fan-out. Go Telegram bot xử lý lệnh /link, /subscribe, /unsubscribe, /status và gửi outbound notification. Hai thành phần này là bề mặt người dùng, vì vậy cần ưu tiên authorization, lỗi rõ ràng và khả năng quan sát trạng thái kết nối.

3 Catalog công nghệ và quyết định lựa chọn

Một báo cáo kỹ thuật tốt không chỉ liệt kê stack công nghệ, mà phải giải thích tại sao mỗi công nghệ được đặt đúng vị trí trong kiến trúc. Với LogRider, stack hiện tại có nhiều service, do đó có vẻ nặng nề nhìn như một danh sách container. Tuy nhiên, khi phân tách theo workload, từng lựa chọn đều phục vụ một ràng buộc khác nhau: broker xử lý burst, pipeline xử lý routing, kho dạng cột xử lý query log, Redis xử lý trạng thái nóng, PostgreSQL xử lý identity, web server xử lý API/UI, worker Python xử lý ML, worker alert xử lý dedup và bot Go xử lý tích hợp Telegram. Cách tổ chức này đánh đổi sự đơn giản triển khai lấy tính tách biệt trách nhiệm. Đối với một hệ thống monitoring, đánh đổi đó hợp lý hơn việc gom toàn bộ xử lý vào một process duy nhất, vì mỗi thành phần có profile tải và rủi ro lỗi khác nhau.

3.1 Redpanda và Pandaproxy

Cặp Redpanda và Pandaproxy tạo thành boundary đầu vào cho log event trong phiên bản hiện tại. Phần này phân tích broker và REST proxy như hai quyết định liên quan nhưng không đồng nhất: broker giải quyết bài toán đệm và phân phối, còn proxy giải quyết bài toán tích hợp nhanh cho producer demo.

3.1.1 Vai trò trong kiến trúc

Redpanda giữ vai trò event backbone của hệ thống. Về mặt thiết kế, đây là quyết định trung tâm vì log là dòng dữ liệu append-only có thể tăng đột biến theo sự cố. Nếu producer ghi trực tiếp vào ClickHouse hoặc gọi trực tiếp classifier, tốc độ của producer sẽ bị ràng buộc bởi component chậm nhất ở phía sau. Broker phá vỡ ràng buộc đó bằng cách biến log thành một chuỗi sự kiện có thể được nhiều consumer đọc độc lập. Điều này cũng tạo ra khả năng replay, đo lag và bổ sung nhánh xử lý mới mà không sửa producer.

3.1.1.1 Lý do không dùng queue đơn giản hơn

Một queue thông thường có thể đủ cho demo nhỏ, nhưng log pipeline cần nhiều nhánh tiêu thụ: unified pipeline, classifier, alert worker, persistence path, DLQ và có thể thêm analytics worker sau này. Kafka-compatible topics làm ranh giới dữ liệu rõ hơn so với một queue generic vì mỗi topic biểu diễn một contract trong pipeline. Khi hệ thống lỗi, kỹ sư có thể kiểm tra topic nào đang lag, topic nào có DLQ và topic nào bị thiếu consumer.

3.1.1.2 Quyết định sử dụng Pandaproxy

Pandaproxy được dùng để producer gửi HTTP request theo Kafka REST format. Đây là lựa chọn thực dụng cho mini-project vì script kiểm thử và producer bên ngoài không cần client Kafka native. Tuy nhiên, Pandaproxy không nên được xem là ingestion API production. Một ingestion API hoàn chỉnh cần kiểm tra API key, schema, quota, application ownership và trả lỗi nghiệp vụ rõ ràng. Vì vậy báo cáo mô tả Pandaproxy như ingress boundary của bản demo, đồng thời xác định ingestion API riêng là hướng phát triển cần ưu tiên.

3.2 Benthos cho routing, normalization và persistence

Benthos được dùng ở nhiều vị trí trong pipeline nên cần được phân tích như một lớp xử lý dữ liệu chuyên biệt. Các tiểu mục bên dưới làm rõ vì sao routing, normalization và persistence được cấu hình bằng pipeline declarative thay vì viết toàn bộ trong server ứng dụng.

3.2.1 Lý do lựa chọn Benthos

Benthos phù hợp với phần pipeline vì phần lớn logic hiện tại là data plumbing: đọc topic, mapping payload, filter theo level, publish sang topic khác, publish Redis pub/sub và ghi HTTP batch vào ClickHouse. Những tác vụ này không cần một service custom phức tạp nếu có thể diễn đạt bằng cấu hình khai báo. Lợi ích của cách làm này là pipeline dễ audit: nhìn vào file YAML có thể thấy input, processor, output, fallback và batch policy.

3.2.1.1 Unified pipeline như content-based router

`pipelines/unified.yaml` vừa chuẩn hóa payload vừa route theo nội dung. Quyết định đặt route ERROR/CRITICAL ở đây giúp alert path nhận tín hiệu sớm, không phải chờ classifier. Đây là một quyết định có tính vận hành: trong incident, latency cảnh báo quan trọng hơn việc log đã có tag ML hay chưa. Đồng thời unified pipeline sinh `Trace_ID` ngay từ đầu để mọi lifecycle event về sau có thể cập nhật cùng một dòng trên dashboard.

3.2.1.2 Persist và tags pipeline như bulk-writer

`pipelines/persist.yaml` và `pipelines/tags.yaml` đặt trách nhiệm ghi ClickHouse ra khỏi worker classifier. Đây là quyết định tốt vì classifier nên tập trung vào inference và enrichment, còn persistence nên tối ưu batch, retry và fallback. ClickHouse thường hoạt động hiệu quả hơn với batch insert, nên việc để Benthos gom batch rồi ghi `JSONEachRow` giúp giảm số insert nhỏ. Rủi ro của lựa chọn này là semantics của trạng thái `Persisted` phải được định nghĩa thật chặt: tốt nhất trạng thái này chỉ nên phát sau khi batch insert thành công, nếu không UI có thể hiển thị một log đã persisted trong khi ClickHouse chưa ghi xong.

3.3 ClickHouse cho lưu trữ log analytics

ClickHouse là quyết định storage quan trọng nhất đối với khả năng phân tích log. Phần này xem xét đặc tính dữ liệu log, pattern truy vấn dashboard và cách schema hiện tại phục vụ hoặc hạn chế các pattern đó.

3.3.1 Đặc tính phù hợp với log

ClickHouse phù hợp vì log enriched là dữ liệu append-only, đọc nhiều hơn sửa và thường được truy vấn theo thời gian, application, level hoặc trace. Thiết kế dạng cột giúp các truy vấn chỉ đọc những cột cần thiết, ví dụ dashboard có thể đọc `Timestamp`, `Application_Name`, `Log_Level`, `Trace_ID` mà không phải xử lý toàn bộ payload lớn. TTL bảy ngày trong schema hiện tại cũng phản ánh đúng đặc tính log demo: giữ dữ liệu đủ để quan sát, nhưng tránh tích lũy vô hạn.

3.3.1.1 Ordering key và workload thực tế

Schema hiện order theo (`Timestamp`, `Trace_ID`). Quyết định này hợp lý nếu truy vấn chủ yếu là recent logs hoặc tìm theo trace trong một khoảng thời gian. Tuy nhiên, nếu workload phổ biến là lọc theo application và level, ordering key có thể chưa tối ưu. Vì vậy báo cáo không khẳng định schema hiện tại là tối ưu cuối cùng; nó là một baseline cần được kiểm chứng bằng query profile thật. Một kỹ sư vận hành nên đo các truy vấn như recent errors by app, recent logs by level, trace lookup và hourly error rate trước khi quyết định đổi ordering key.

3.3.1.2 *Materialized view cho health metrics*

hourly_health_mv tổng hợp error_count và total_count theo giờ/application. Đây là quyết định quan trọng vì dashboard không nên scan bảng log đầy đủ mỗi khi vẽ chart error rate. Dù vậy, aggregate hiện tại mới trả lời được câu hỏi ở granularity giờ. Nếu dashboard cần các cửa sổ 1 phút, 5 phút hoặc theo environment, schema aggregate cần mở rộng.

3.4 Redis cho runtime state, pub/sub và deduplication

Redis xuất hiện ở nhiều nhánh runtime nên cần phân biệt rõ vai trò. Một số dữ liệu trong Redis là tín hiệu pub/sub ngắn hạn, một số là state dùng cho deduplication, một số là queue hoặc session tạm thời; nhầm lẫn giữa các loại này sẽ dẫn đến sai quyết định persistence.

3.4.1 *Vai trò đa nhiệm*

Redis được dùng ở nhiều vị trí: pub/sub lifecycle event, WebSocket fan-out, dedup state cho alert, Telegram link/session và outbound queue. Điểm chung của các dữ liệu này là chúng cần tốc độ truy cập thấp-latency và phần lớn có tính transient. Đặt các trạng thái này vào ClickHouse sẽ sai workload vì ClickHouse không phù hợp với update nhỏ theo key. Đặt vào PostgreSQL cũng làm tăng áp lực transaction cho dữ liệu không cần bền vững dài hạn.

3.4.1.1 *Trade-off về tính bền vững*

Lựa chọn Redis luôn đi kèm câu hỏi: dữ liệu mất thì hậu quả là gì. Nếu mất pub/sub event, dashboard live có thể bỏ lỡ một trạng thái nhưng vẫn có thể hydrate lại từ ClickHouse. Nếu mất dedup state, hệ thống có thể gửi lại alert như lỗi mới. Nếu mất Telegram session, người dùng có thể phải link lại. Vì vậy cần phân loại rõ state nào chấp nhận transient, state nào cần persist hoặc checkpoint. Ở phiên bản benchmark này, Redis được xác minh cho session token, stream alert, notifications:data/notifications:index, dedup:global:* và telegram_outbound, nhưng chưa có audit store dài hạn cho alert delivery.

3.4.1.2 *Lua và atomic update*

Alert deduplication là một bài toán read-modify-write. Nếu nhiều alert giống nhau đến cùng lúc, update count bằng nhiều lệnh Redis độc lập có thể gặp race condition. Dùng Lua hoặc pipeline atomic giúp gom logic update vào một thao tác nhất quán hơn. Đây là ví dụ điển hình của quyết định kỹ thuật nhỏ nhưng ảnh hưởng lớn tới correctness: nếu counter sai, notification policy cũng sai.

3.5 PostgreSQL cho user và RBAC

PostgreSQL được dùng cho phần identity và phân quyền vì đây là nhóm dữ liệu cần consistency và khả năng truy vấn quan hệ. Phần này giải thích vì sao user/RBAC không nên bị trộn vào ClickHouse dù ClickHouse đang lưu log chính.

3.5.1 *Lý do tách identity khỏi log store*

User, role và allowed application là dữ liệu transactional. Chúng cần uniqueness, update an toàn, password hashing và khả năng truy vấn theo username. ClickHouse không phải công cụ phù hợp cho dữ liệu dạng này. PostgreSQL đáp ứng tốt hơn vì nó cung cấp consistency, index, constraint và mô hình dữ liệu dễ mở rộng. Quyết định dùng PostgreSQL cho identity giúp không trộn log analytics với dữ liệu xác thực.

3.5.1.1 Giới hạn của mô hình *allowed_apps* dạng text

Schema hiện tại lưu *allowed_apps* dạng text. Cách này nhanh cho demo nhưng không lý tưởng khi hệ thống có nhiều team, organization hoặc permission phức tạp. Một thiết kế trưởng thành hơn nên tách bảng *users*, *applications*, *user_application_grants* và audit các thay đổi quyền. Điểm quan trọng là báo cáo không che giấu giới hạn này; nó xem đây là bước tiếp theo của RBAC hardening.

3.6 Bun web server và frontend tĩnh

Web server là boundary giữa dữ liệu backend và trải nghiệm vận hành của người dùng. Các tiểu mục dưới đây phân tích việc gộp HTML serving, REST API và WebSocket vào một service Bun, đồng thời chỉ ra điểm cần gia cố ở authorization cho live stream.

3.6.1 Quyết định gộp HTML, REST API và WebSocket

Bun server hiện phục vụ HTML pages, REST APIs, authentication và WebSocket upgrade. Với một mini-project, gộp các vai trò này giúp giảm chi phí triển khai và đơn giản hóa demo. Server là nơi tự nhiên để kiểm tra token, đọc PostgreSQL, query ClickHouse và subscribe Redis. Lựa chọn này làm cho luồng người dùng ngắn: đăng nhập, tải historical data, mở WebSocket và nhận event live.

3.6.1.1 Rủi ro khi hệ thống lớn hơn

Khi traffic tăng, gộp web/API/WebSocket có thể tạo bottleneck. WebSocket connection sống lâu, query ClickHouse có thể nặng, còn authentication phải ổn định. Nếu tất cả nằm trong một process, lỗi ở một phần có thể ảnh hưởng phần còn lại. Một hướng phát triển hợp lý là tách API server, WebSocket gateway và static frontend, nhưng điều đó chỉ nên làm khi số liệu tải chứng minh cần thiết.

3.6.1.2 Server-side authorization cho stream

Historical API có thể lọc bằng query ClickHouse. WebSocket khó hơn vì event đi vào từ Redis pub/sub liên tục. Do đó server phải kiểm tra role/app scope trước khi forward từng event. Đây là điểm khác biệt giữa bảo mật UI và bảo mật backend: ẩn dòng log trên frontend không đủ nếu backend đã gửi event nhạy cảm tới client.

3.7 Python classifier worker

Classifier worker thể hiện quyết định tách xử lý ML khỏi pipeline chuẩn hóa và lưu trữ. Phần này giải thích lợi ích của việc cô lập inference, đồng thời nêu các câu hỏi đánh giá model còn thiếu trong dữ liệu đầu vào.

3.7.1 Lý do dùng Python và ONNX Runtime

Python là lựa chọn tự nhiên cho classifier vì hệ sinh thái NLP, Hugging Face và ONNX Runtime trưởng thành. Đặt classifier thành worker riêng là quyết định đúng vì inference có đặc tính CPU/memory khác với API server. Worker có thể batch message, tối ưu model runtime và scale độc lập. Nếu inference chậm, broker sẽ thể hiện lag ở topic tương ứng thay vì làm web server treo.

3.7.1.1 Chất lượng classifier cần được đo độc lập

Một classifier chỉ có giá trị khi có bộ dữ liệu đánh giá. Benchmark cục bộ xác nhận worker tạo tag cho 100/100 log trong 7.203 giây, tương đương khoảng 13.88 log/giây end-to-end tới bảng *log_tags*. Tuy nhiên báo cáo chưa có dataset có nhãn, precision, recall, F1 hoặc confusion matrix. Vì vậy có thể kết luận classifier path hoạt động và tạo tag, nhưng chưa thể kết luận chất lượng phân loại theo nghĩa machine learning.

3.8 Node/Bun alert worker và Go Telegram bot

Nhánh alert có yêu cầu khác với nhánh lưu trữ log: độ trễ, deduplication, scope người nhận và khả năng tránh nhiễu đều quan trọng. Vì vậy, alert worker và Telegram bot được phân tích như một subsystem notification có state và policy riêng.

3.8.1 Alert worker như policy engine

Alert worker dùng JavaScript/Node/Bun và KafkaJS để consume `alerts-ingested`. Logic của worker chủ yếu là I/O-bound: đọc Kafka, tính signature, cập nhật Redis, publish alert event và push job Telegram. Vì vậy lựa chọn Node/Bun là thực dụng. Điểm quan trọng hơn ngôn ngữ là worker phải đóng vai trò policy engine: nó không chỉ gửi mọi lỗi, mà quyết định lỗi nào là mới, lỗi nào đạt threshold và lỗi nào chỉ cập nhật count âm thầm.

3.8.1.1 Telegram bot như delivery adapter

Telegram bot dùng Go cho một service nhỏ, chạy lâu dài, ít dependency và xử lý lệnh rõ ràng. Bot không nên chứa logic dedup hay RBAC phức tạp; nó chỉ nên xác minh token link, lưu chat session và gửi outbound message. Tách bot khỏi alert worker giúp kênh delivery có thể thay đổi sau này, ví dụ email, Slack hoặc webhook, mà không viết lại policy alert.

3.9 Docker Compose cho môi trường demo

Docker Compose là cách đóng gói phù hợp cho báo cáo mini-project nhiều dependency. Phần này làm rõ Compose hỗ trợ tái lập demo như thế nào, đồng thời không đánh đồng khả năng chạy local với mức sẵn sàng production.

3.9.1 Lý do chọn Docker Compose

Docker Compose phù hợp với báo cáo mini-project vì hệ thống gồm nhiều dependency: Redpanda, Redis, ClickHouse, PostgreSQL, Benthos pipelines, workers, web server và bot. Compose cho phép người đánh giá chạy lại stack bằng một entrypoint thống nhất, đồng thời làm rõ quan hệ service trong một file cấu hình. Trong bối cảnh học tập và demo, tính tái lập quan trọng hơn việc triển khai production ngay.

3.9.1.1 Giới hạn production

Compose không cung cấp autoscaling, rolling update, secret management mạnh, service mesh, persistent volume policy hoặc high availability theo cụm. Vì vậy báo cáo chỉ claim hệ thống có thể chạy và kiểm thử end-to-end trong Docker Compose. Các yêu cầu production như Kubernetes, CI/CD, secret rotation, multi-node benchmark và chaos testing được giữ ở phần hướng phát triển.

3.10 Tổng hợp trade-off công nghệ

Nhìn tổng thể, LogRider chọn kiến trúc nhiều thành phần vì mỗi thành phần có một workload khác nhau. Điểm mạnh là pipeline rõ ràng, dễ thay thế từng stage và phù hợp với tư duy event-driven. Điểm yếu là vận hành phức tạp hơn một monolith: nhiều container, nhiều topic, nhiều state store và nhiều cấu hình phân tán. Đây là trade-off chấp nhận được ở một hệ thống monitoring nếu báo cáo luôn nói rõ phạm vi prototype: đã có benchmark cục bộ và test end-to-end, nhưng chưa có hardening, observability và benchmark production nhiều node.

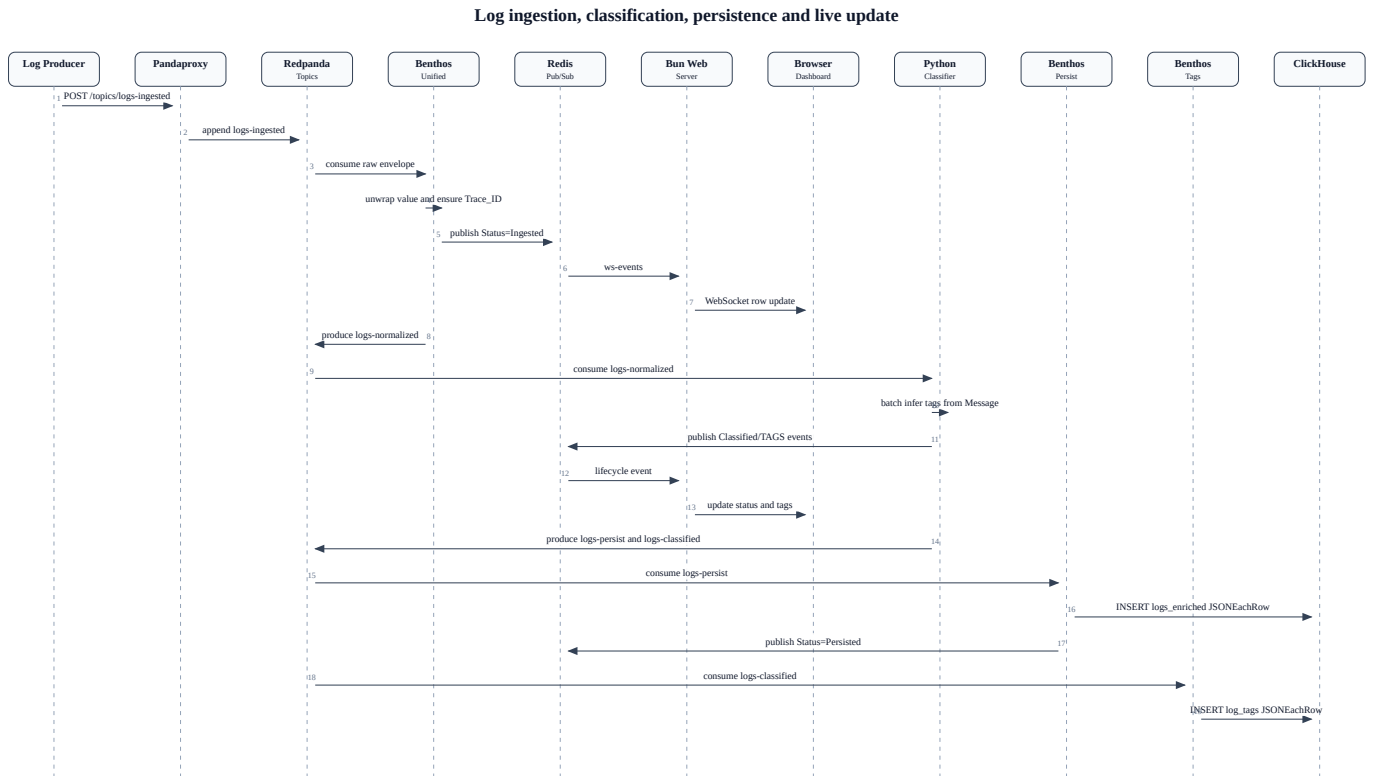
4 Thiết kế luồng dữ liệu và sequence diagram

Phần này chuyển từ kiến trúc tĩnh sang hành vi động. Các sequence diagram được trình bày theo phong cách PlantUML với lifeline rõ ràng để người đọc thấy một event đi qua hệ thống như thế nào, component nào gọi

component nào và state nào được cập nhật ở từng bước.

4.1 Luồng ingest, classify và persist

Hình 2 mô tả sequence chính từ producer đến ClickHouse và dashboard dưới dạng sơ đồ sequence theo phong cách PlantUML, với lifeline tách biệt cho từng actor. Các nhãn được đặt trên mũi tên thay vì trong bảng để người đọc nhìn thấy thứ tự thời gian và chiều trao đổi message.

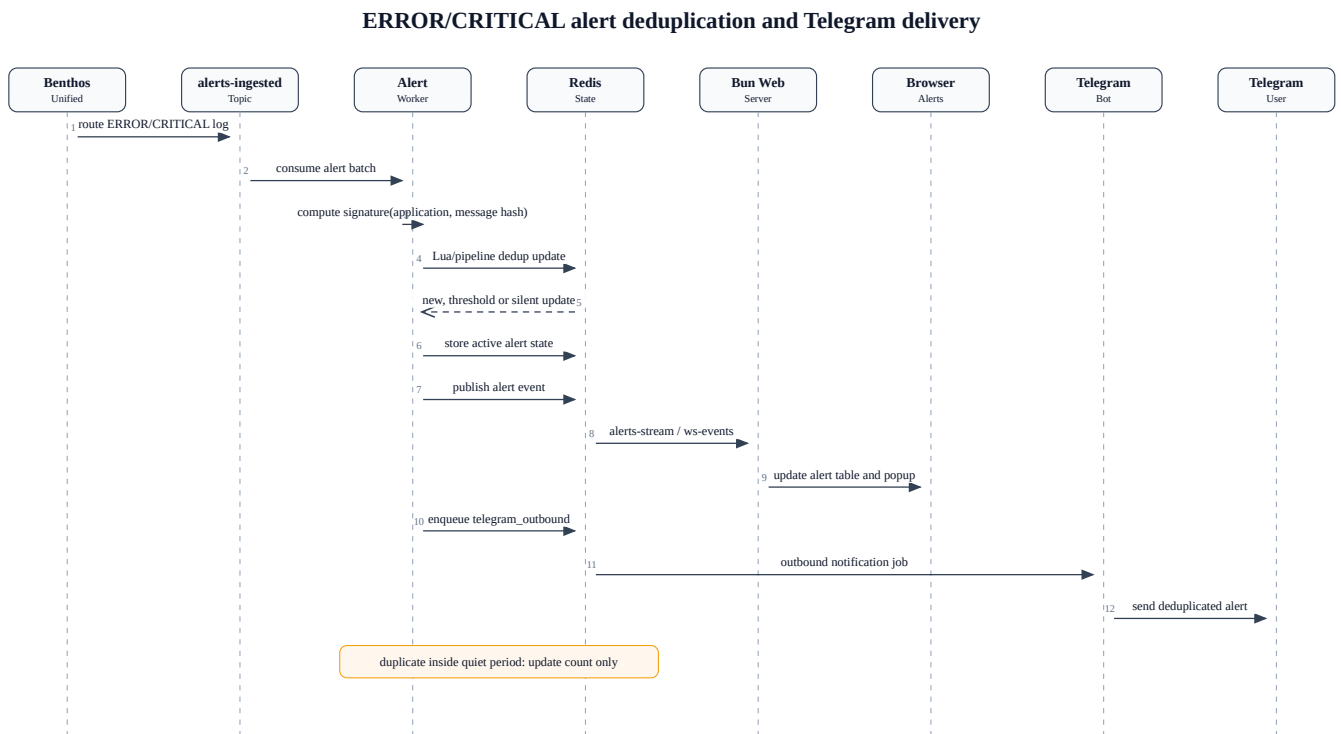


Hình 2: Sequence diagram luồng tiếp nhận, phân loại và lưu trữ log

Producer gửi payload vào Pandaproxy. Redpanda ghi vào logs-ingested. Benthos unified tiêu thụ topic này, chuẩn hóa trường value, sinh Trace_ID nếu thiếu và phát trạng thái Ingested. Sau đó, unified produce log đã chuẩn hóa vào logs-normalized. Classifier worker tiêu thụ theo batch, suy luận nhãn và produce bản ghi enriched. Benthos persist nhận batch, ghi vào ClickHouse và fallback sang DLQ nếu HTTP insert thất bại. Luồng này phù hợp với nguyên tắc pipeline hóa dữ liệu và tách consumer theo trách nhiệm [3, 1].

4.2 Luồng cảnh báo và deduplication

Hình 3 trình bày luồng cảnh báo bằng sequence diagram theo phong cách PlantUML. Điểm quan trọng của sơ đồ là alert không đi thẳng từ unified tới Telegram; alert worker có một bước trung gian dùng Redis để quyết định action và phát trạng thái.



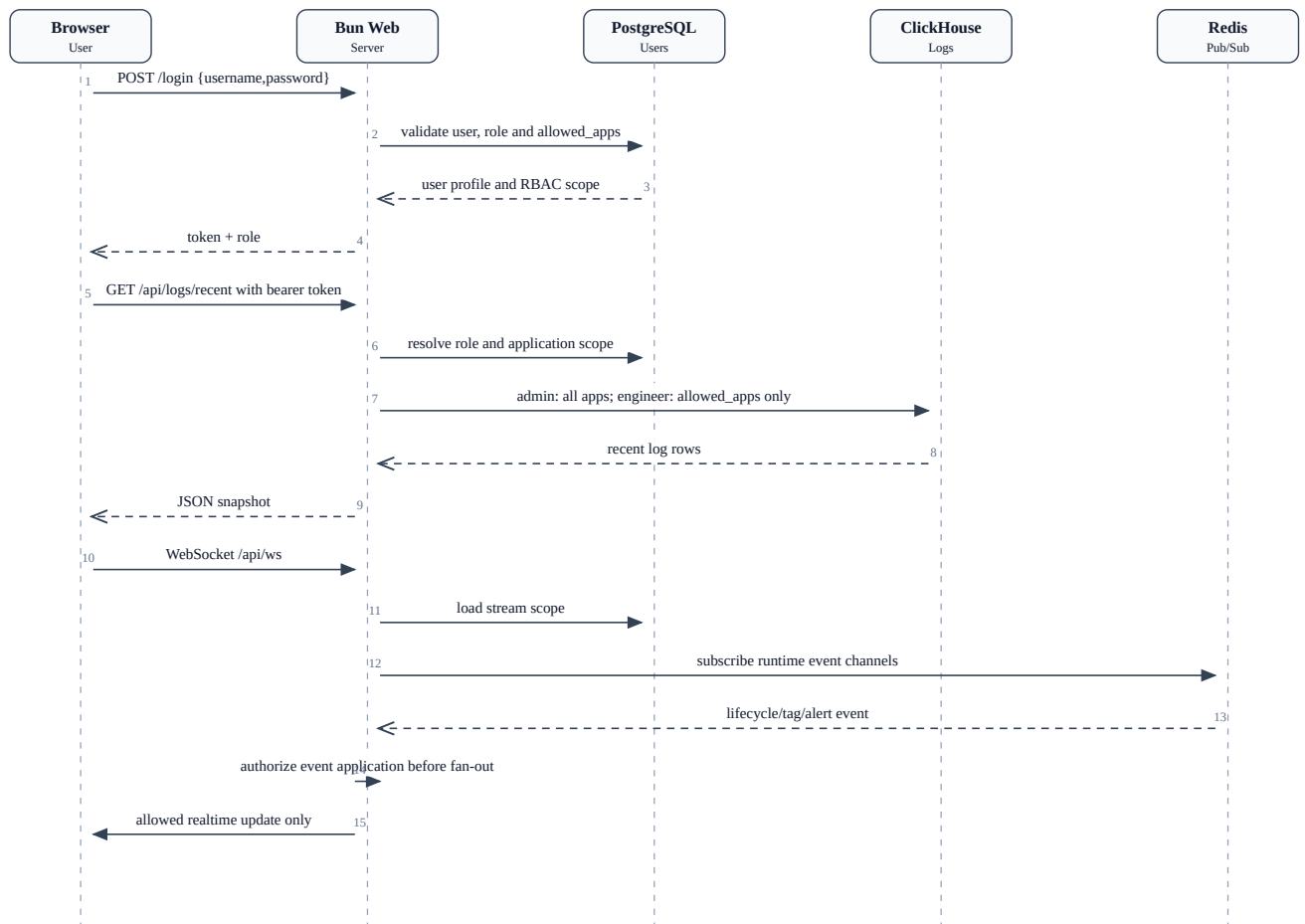
Hình 3: Sequence diagram luồng cảnh báo, deduplication và notification

Nếu action là `new`, hệ thống thông báo ngay lần đầu. Nếu action là `threshold`, hệ thống thông báo lại tại các mốc số lần xuất hiện. Điều này cần được diễn giải rõ trong requirement. Trong vận hành thực tế, `threshold` reminder có thể hữu ích khi sự cố tăng mạnh, nhưng cũng có thể gây noise nếu requirement mong muốn suppress toàn bộ lỗi lặp lại trong một cửa sổ thời gian.

4.3 Luồng authentication, API và RBAC

Hình 4 mô tả luồng truy vấn lịch sử log có xác thực bằng sequence diagram theo phong cách PlantUML. Web server lấy token từ header hoặc cookie, xác định user/role, sau đó áp dụng filter khi truy vấn ClickHouse. Phần này cần được phân biệt với live stream WebSocket: API có thể lọc theo truy vấn, còn WebSocket phải lọc từng event trước khi gửi.

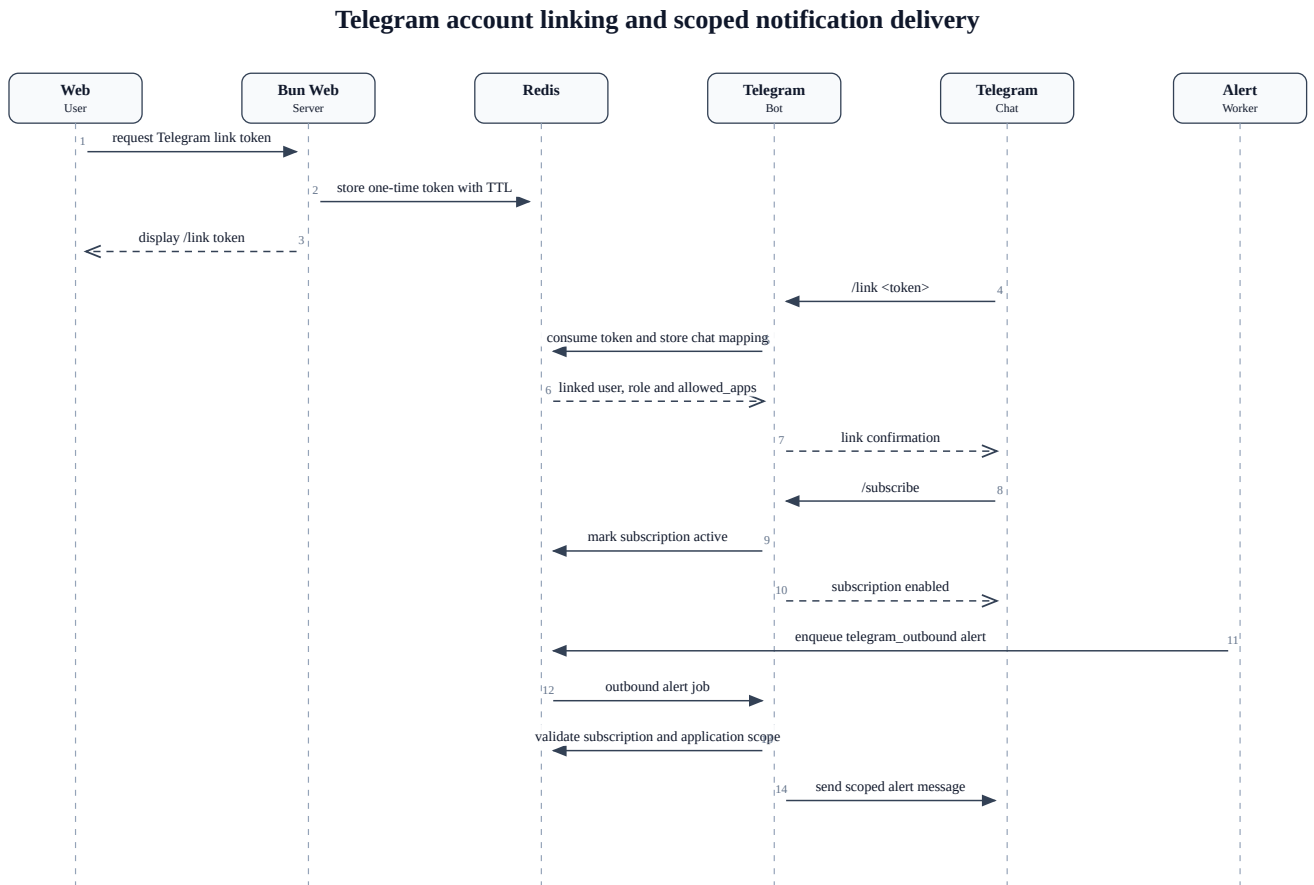
Authentication, RBAC-scoped historical query and live stream filtering



Hình 4: Sequence diagram theo phong cách PlantUML cho authentication và truy vấn log có RBAC

4.4 Luồng liên kết Telegram

Hình 5 mô tả cách người dùng liên kết tài khoản LogRider với Telegram chat bằng sequence diagram theo phong cách PlantUML. Token liên kết có TTL ngắn, được sinh từ web server và tiêu thụ bởi bot khi người dùng gửi lệnh /link. Cách thiết kế này tránh việc người dùng phải nhập credential vào Telegram bot.



Hình 5: Sequence diagram theo phong cách PlantUML cho liên kết Telegram và gửi thông báo

5 Thiết kế dữ liệu

Thiết kế dữ liệu xác định hợp đồng giữa producer, pipeline, worker, storage và dashboard. Phần này không chỉ liệt kê cột, topic hoặc Redis key, mà còn giải thích vai trò vận hành của từng nhóm dữ liệu và hậu quả khi schema không ổn định.

5.1 Schema log canonical

Schema log là hợp đồng giữa producer, pipeline, classifier, storage và dashboard. Nếu field không ổn định, các tầng sau sẽ phải xử lý nhiều nhánh đặc biệt và khó tối ưu truy vấn. Bảng 2 mô tả schema canonical được suy ra từ ClickHouse và pipeline.

Bảng 2: Schema log canonical và ý nghĩa vận hành

Trường	Kiểu/giá trị	Ý nghĩa
Application_Name	String	Tên ứng dụng hoặc service phát sinh log; dùng cho lọc dashboard, RBAC, group alert và metrics theo application.
Log_Level	Enum-like string	Một trong DEBUG, INFO, WARN, ERROR, CRITICAL. ClickHouse lưu bằng Enum8.

Trường	Kiểu/giá trị	Ý nghĩa
Message	String	Nội dung log; dùng cho hiển thị, classification và hash deduplication.
Timestamp	DateTime64(3)	Thời điểm phát sinh log; là cột quan trọng cho timeline, TTL và materialized view theo giờ.
Trace_ID	UUID	Mã truy vết. Pipeline sinh UUID khi thiếu để mọi bản ghi có khóa dùng cho cập nhật trạng thái UI.
Tags	Array(String)	Nhãn enrich từ classifier; chỉ có ở bảng enriched hoặc log tags.
Status	Transient string	Trạng thái live như Ingested, Persisted; không phải cột bền vững trong bảng ClickHouse chính.

5.2 Catalog topic và trách nhiệm

Bảng 3 trình bày topic chính, producer và consumer tương ứng. Bảng này hữu ích khi viết test tích hợp vì mỗi topic phải có cách seed dữ liệu, consumer mong đợi và output để kiểm tra.

Bảng 3: Catalog topic trong pipeline LogRider

Topic	Producer chính	Consumer và vai trò
logs-ingested	Pandaproxy hoặc producer demo	Benthos unified tiêu thụ payload thô, unwrap trường value, sinh Trace_ID nếu thiếu và phát trạng thái Ingested.
logs-normalized	Benthos unified	Python classifier tiêu thụ log đã chuẩn hóa để phân loại message và phát enriched log sang nhánh persistence.
logs-persist	Python classifier hoặc processing path	Benthos persist batch insert vào ClickHouse bảng logs_enriched; khi lỗi ghi cần fallback sang DLQ.
logs-classified	Python classifier	Benthos tags ghi bản ghi tag vào ClickHouse bảng log_tags, phục vụ truy vấn tag độc lập theo Trace_ID.
alerts-ingested	Benthos unified	Alert worker và alert forwarder tiêu thụ log ERROR/CRITICAL; worker thực hiện deduplication và đẩy notification state.
dlq-clickhouse	Benthos persist fallback	Dành cho bản ghi không ghi được vào ClickHouse; cần quy trình replay và dashboard DLQ trong các phiên bản sau.
dlq-logs	Benthos tags fallback	Dành cho tag record lỗi; cần metric số lượng DLQ để phát hiện classifier/persistence mismatch.

5.3 ClickHouse và PostgreSQL

ClickHouse đảm nhận lưu trữ log analytics. Bảng logs_enriched là bảng quan trọng nhất với engine MergeTree, ordering key (Timestamp, Trace_ID) và TTL 7 ngày. Bảng hourly_health_mv dùng AggregatingMergeTree để lưu error_count và total_count theo giờ và application. PostgreSQL hiện giữ bảng users. Bảng 4 tổng hợp nhóm dữ liệu bền vững.

Bảng 4: Nhóm dữ liệu bền vững trong ClickHouse/PostgreSQL

Bảng	Engine / hệ CSDL	Vai trò
logrider.logs_enriched	ClickHouse MergeTree	Lưu log enriched cùng Tags. Đây là nguồn chính cho dashboard và query lịch sử.
logrider.logs	ClickHouse MergeTree	Bảng log không có tags; cần xác minh còn được dùng trong server/UI hay là di sản schema.
logrider.log_tags	ClickHouse MergeTree	Lưu tags theo Trace_ID. Pipeline tags.yaml ghi từ logs-classified.
logrider.hourly_health_mv	ClickHouse Aggregating MergeTree	Lưu thống kê error/total theo giờ và application cho charts.
users	PostgreSQL	Lưu username, password_hash, role và allowed_apps phục vụ authentication/RBAC.

5.4 Redis runtime state

Redis được dùng cho trạng thái tốc độ cao và dữ liệu ngắn hạn. Bảng 5 phân loại các nhóm key/channel theo mục đích. Vì Redis trong thiết kế này chứa runtime state, các dữ liệu cần audit dài hạn không nên chỉ nằm trong Redis.

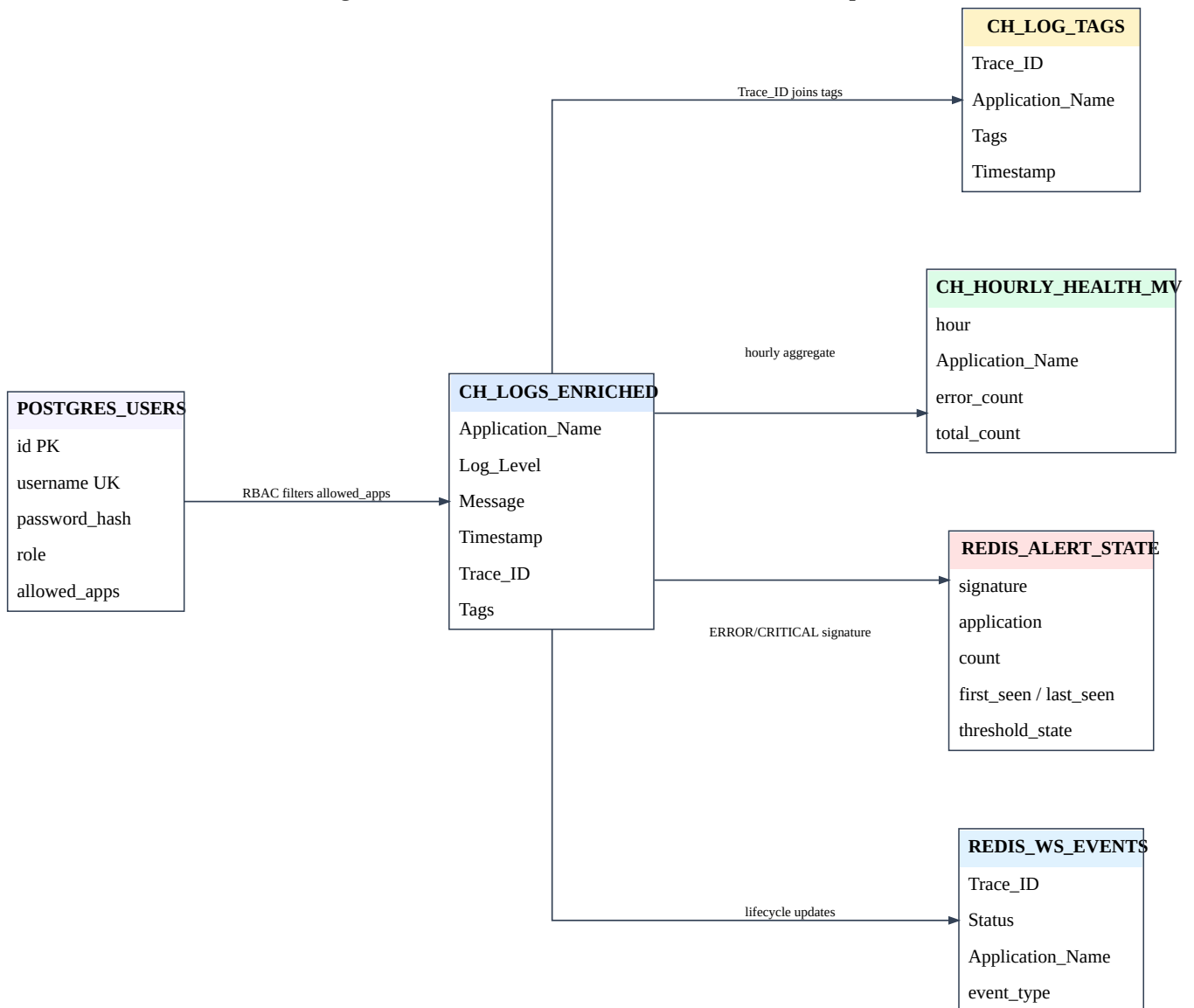
Bảng 5: Nhóm trạng thái Redis trong LogRider

Nhóm	Ví dụ key/channel	Vai trò
WebSocket event	ws-events	Pub/sub trạng thái log và tag cho browser.
Alert stream	alerts-stream	Pub/sub alert event cho web server/dashboard.
Dedup state	dedup:global:{Application_Name}, config:dedup	Lưu config dedup, last seen và threshold state cho alert worker.
Recent notifications	notifications:data, notifications:index	Lưu trạng thái alert gần đây phục vụ API hoặc UI.
Telegram link token	link_token:{token} hoặc token TTL tương đương trong Redis	Liên kết user LogRider với Telegram chat trong thời gian ngắn.
Telegram outbound	telegram_outbound	Queue job gửi Telegram notification.
Session/transient config	session:{token}, config:alert_ttl, config:noti_ttl	Phục vụ trạng thái đăng nhập hoặc cấu hình runtime trong web server.

5.5 Sơ đồ dữ liệu rút gọn

Hình 6 mô tả mô hình dữ liệu rút gọn. Sơ đồ không liệt kê toàn bộ cột kỹ thuật mà tập trung vào mối quan hệ giữa log event, trace, application, tags, alert state và user/RBAC.

LogRider data model and runtime-state relationships



Hình 6: Mô hình dữ liệu rút gọn của log, alert và RBAC

6 Thiết kế API, giao diện và quyền truy cập

Bề mặt người dùng của LogRider gồm dashboard, alerts, metrics và config. Web server Bun cung cấp các route HTML và API. Bảng 6 mô tả nhóm chức năng ở tầng serving. Các endpoint cụ thể có thể thay đổi theo code, nên bảng tập trung vào vai trò nghiệp vụ.

Bảng 6: Nhóm giao diện và API serving

Nhóm	Bề mặt	Vai trò
Authentication	Login form, token/cookie	Cho phép user đăng nhập, nhận token và sử dụng API/dashboard.

Nhóm	Bề mặt	Vai trò
Dashboard	/dashboard, recent logs API, WebSocket	Hiển thị log gần đây, trạng thái pipeline và cập nhật live.
Alerts	/alerts, alert APIs	Hiển thị alert stream, recent alerts và popup/notification state.
Metrics	/metrics	Hiển thị error rate theo thời gian và ranking application theo lỗi.
Config	/config	Dành cho admin cấu hình một số thông tin runtime. Cần rà soát quyền truy cập và persistence.
Telegram link	API generate link token	Sinh token TTL ngắn để user liên kết chat Telegram với tài khoản.
WebSocket	Upgrade từ browser	Fan-out Redis pub/sub event tới UI; cần lọc application theo RBAC cho non-admin.

Bảng 7 mô tả ma trận quyền dự kiến. Cột “rủi ro hiện tại” nhấn mạnh phần live stream vì đây là điểm khác biệt giữa truy vấn lịch sử có filter và stream realtime.

Bảng 7: Ma trận RBAC dự kiến cho dashboard và stream

Role	Historical API	Live WebSocket	Rủi ro/cần xử lý
Admin	Xem toàn bộ application và alerts	Nhận global stream	Phù hợp với vai trò quản trị, cần audit thao tác nhạy cảm.
Engineer	Chỉ xem application trong <code>allowed_apps</code>	Subscribe <code>ws-frontend:{app}</code> theo allowed apps; không nhận global log stream	Cần tiếp tục test negative case nhiều application và phân biệt global alert với app-scoped log.
Unauthenticated	Không được xem API dữ liệu	Không được mở stream dữ liệu	Cần bảo đảm token/cookie được kiểm tra trước khi upgrade.
Telegram chat	Nhận alert theo subscription đã link	Không áp dụng	Cần kiểm tra token TTL, unlink/revoke và role/application scope.

7 Phân tích quyết định thiết kế theo góc nhìn kỹ sư

Phần này tập trung vào reasoning đằng sau các quyết định quan trọng. Một báo cáo kỹ thuật tốt cần cho thấy tác giả hiểu vì sao hệ thống được thiết kế như vậy, lựa chọn nào đã được ưu tiên và rủi ro nào được chấp nhận trong bối cảnh prototype.

7.1 Tại sao không ghi log trực tiếp vào storage

Quyết định không ghi trực tiếp vào storage xuất phát từ nhu cầu bảo vệ producer và mở rộng pipeline. Các tiểu mục dưới đây phân tích quyết định này từ hai phía: ràng buộc của producer đang chạy production path và ràng buộc của storage khi nhận lượng insert lớn.

7.1.1 Ràng buộc producer

Từ góc nhìn producer, log path phải có chi phí thấp và không được trở thành điểm làm chậm business request. Vì vậy, phân tích producer tập trung vào độ trễ, dependency ngược và khả năng bổ sung nhánh xử lý mà không sửa mã ứng dụng gửi log.

7.1.1.1 Độ trễ và độ phụ thuộc

Nếu producer ghi trực tiếp vào ClickHouse, mọi sự cố ở storage sẽ lan ngược về application. Điều này làm log path trở thành một dependency runtime của business path, trong khi log nên là telemetry path có khả năng chịu lỗi. Đưa Redpanda vào giữa giúp producer chỉ cần produce event, còn lỗi ClickHouse được xử lý ở consumer hoặc DLQ.

7.1.1.2 Khả năng thêm nhánh xử lý

Khi log đã nằm trong broker, thêm classifier, alert worker hoặc analytics consumer không cần sửa producer. Đây là lý do kiến trúc event-driven phù hợp với một sản phẩm monitoring đang phát triển: mỗi capability mới nên là một consumer mới hoặc một pipeline branch mới, không phải một thay đổi ở từng application gửi log.

7.1.2 Ràng buộc storage

Từ góc nhìn storage, mục tiêu không chỉ là ghi được một dòng log mà là ghi ổn định dưới tải lớn. Vì vậy, batching, retry và DLQ trở thành các cơ chế cần thiết để giảm áp lực lên ClickHouse.

7.1.2.1 Batching

ClickHouse tối ưu hơn khi nhận batch lớn. Nếu từng producer tự insert từng dòng, hệ thống dễ gặp nhiều insert nhỏ, tốn overhead và khó kiểm soát retry. Benthos persist gom batch và fallback giúp storage path có chính sách riêng.

7.2 Tại sao cần Trace_ID ngay từ stage đầu

Trace identifier được đặt sớm vì toàn bộ pipeline cần một khóa ổn định để liên kết trạng thái. Nếu khóa này chỉ xuất hiện sau khi persist, dashboard không thể cập nhật lifecycle của cùng một log trong thời gian gần thực.

7.2.1 Liên kết lifecycle

Lifecycle của một log trong LogRider gồm nhiều trạng thái tạm thời và bền vững. Do đó, phần này phân tích Trace_ID như một khóa liên kết giữa event normalization, classification, persistence và UI update.

7.2.1.1 Một log, nhiều trạng thái

Cùng một log đi qua các trạng thái Ingested, Classified, Persisted và có thể sinh alert. Nếu không có khóa ổn định, dashboard khó cập nhật cùng một dòng mà không tạo bản ghi trùng. Trace_ID vì vậy là khóa lifecycle tối thiểu.

7.2.1.2 Giới hạn hiện tại

Nếu producer không gửi trace id ứng dụng, pipeline sinh UUID mới. UUID này đủ để liên kết nội bộ trong LogRider, nhưng không giúp truy vết xuyên service. Hướng mở rộng là chấp nhận W3C traceparent hoặc OpenTelemetry trace id nếu producer có sẵn.

7.3 Tại sao tách alert khỏi classifier

Alert và classification phục vụ hai mục tiêu khác nhau. Alert cần phản ứng nhanh với severity thô, trong khi classifier bổ sung ngữ cảnh sâu hơn nhưng có thể chậm hoặc lỗi; tách hai nhánh giúp failure của ML không chặn notification nghiêm trọng.

7.3.1 Độ ưu tiên xử lý

Ưu tiên xử lý trong alert path được xác định bởi khả năng hành động của người vận hành. Phần này giải thích vì sao severity-based routing được đặt ở unified pipeline thay vì đợi toàn bộ enrichment hoàn tất.

7.3.1.1 Alert không nên chờ ML

Log ERROR/CRITICAL cần được đưa vào alert path sớm. Nếu alert phải chờ classifier, một lỗi model hoặc batch inference chậm có thể làm trễ notification. Vì vậy route alert ở unified stage là quyết định hợp lý: alert path dựa trên severity thô, còn tags/classification phục vụ ngữ cảnh bổ sung.

7.3.1.2 Noise và dedup

Tách alert sớm không có nghĩa gửi mọi lỗi ngay lập tức. Alert worker vẫn cần deduplication, threshold và suppression. Điểm cần hoàn thiện là viết rõ semantics và test số notification theo cùng một signature lỗi.

7.4 Tại sao dashboard cần cả lịch sử và stream

Dashboard realtime luôn cần hai nguồn dữ liệu: snapshot ban đầu và delta sau thời điểm kết nối. Nếu thiếu một trong hai nguồn này, người dùng hoặc không thấy lịch sử gần đây, hoặc không thấy trạng thái mới phát sinh.

7.4.1 Hydration ban đầu

Hydration ban đầu là bước tải snapshot từ storage bền vững trước khi áp dụng các event thời gian thực. Điều này giúp giao diện bắt đầu từ trạng thái đầy đủ thay vì chỉ nhìn thấy những gì xảy ra sau khi WebSocket mở.

7.4.1.1 Historical API

Khi người dùng mở dashboard, WebSocket chỉ cho biết event mới phát sinh sau thời điểm kết nối. Vì vậy /api/logs/recent cần hydrate trạng thái ban đầu từ ClickHouse. Đây là mô hình phổ biến của realtime UI: tải snapshot trước, sau đó apply delta từ stream.

7.4.2 Delta thời gian thực

Delta thời gian thực bổ sung các thay đổi mới vào snapshot. Vì delta có thể chứa dữ liệu nhạy cảm tương tự API lịch sử, stream phải áp dụng cùng một chính sách RBAC ở server-side.

7.4.2.1 WebSocket fan-out

Redis pub/sub và WebSocket giúp dashboard cập nhật status/tags/alert gần thời gian thực. Tuy nhiên, stream live phải áp dụng cùng policy RBAC như historical API. Nếu snapshot đã lọc theo allowed_apps nhưng WebSocket lại gửi global event, UI sẽ rò dữ liệu dù API lịch sử đúng.

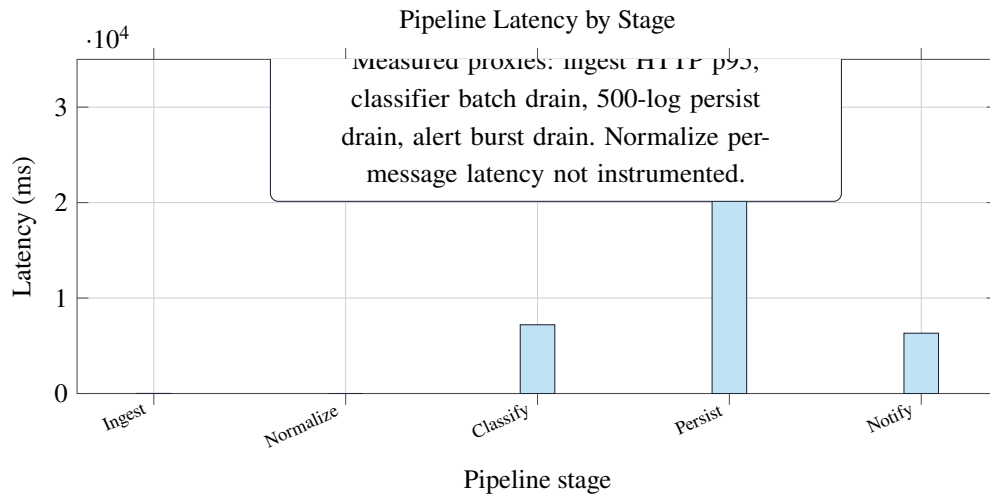
8 Phương pháp kiểm thử và đo lường

Phần kiểm thử được thiết kế theo hai mức. Mức thứ nhất là functional smoke test để xác nhận pipeline chạy đúng từ input tới output. Mức thứ hai là measurement test để đo throughput, latency, error rate và tính đúng của dedup/RBAC. Bảng 8 tổng hợp ma trận kiểm thử đề xuất.

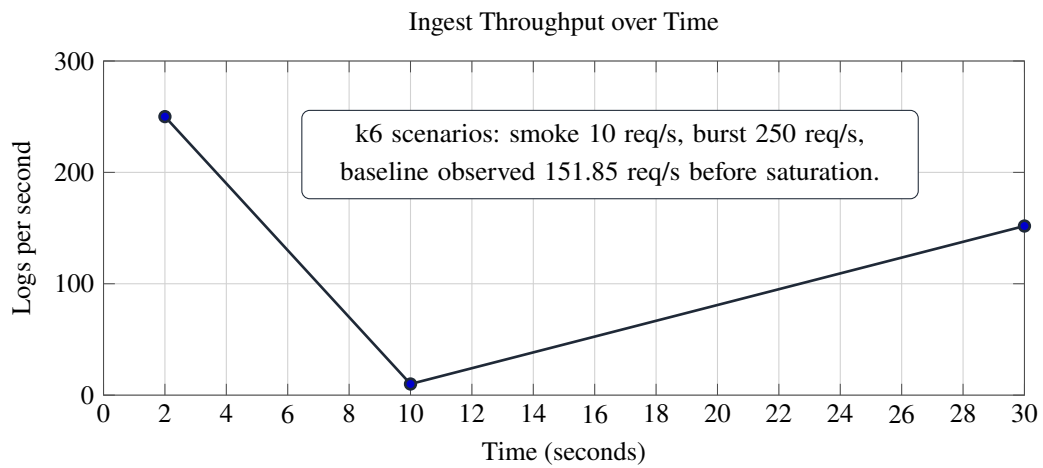
Bảng 8: Ma trận kiểm thử và đánh giá đề xuất

Mã	Mục tiêu	Cách kiểm thử	Kết quả kỳ vọng
TC-01	Khởi động stack	Chạy <code>docker compose up -d</code> , kiểm tra health của Redpanda, Redis, ClickHouse, PostgreSQL, server và worker.	Tất cả service cốt lõi ở trạng thái healthy hoặc running; không restart loop.
TC-02	Ingest log đơn giản	Gửi 5 log tương ứng 5 level bằng payload Kafka REST hợp lệ.	ClickHouse có 5 bản ghi enriched; dashboard nhận Ingested và Persisted.
TC-03	Classification tags	Gửi log có message đại diện cho các nhóm nhãn.	Bản ghi enriched có Tags; Redis/WebSocket phát tag event.
TC-04	Alert routing	Gửi log ERROR và CRITICAL.	Topic <code>alerts-ingested</code> có event; alert worker phát alert stream.
TC-05	Dedup semantics	Gửi 100 lỗi giống nhau trong 60 giây.	Kết quả đo: 100 alert stream update, một key recent notification, một dedup key có count 100; Telegram queue bằng 0 khi chưa có subscriber. Semantics hiện tại là threshold/update, không phải exactly-one WebSocket event.
TC-06	RBAC historical	Đăng nhập admin và engineer, truy vấn log nhiều application.	Engineer chỉ thấy application được cấp; admin thấy toàn bộ.
TC-07	RBAC live stream	Mở WebSocket bằng engineer, gửi log application ngoài phạm vi.	Engineer không nhận event ngoài <code>allowed_apps</code> .
TC-08	ClickHouse failure	Tạm làm ClickHouse không ghi được trong khi gửi log.	Bản ghi lỗi chuyển sang <code>dlq-clickhouse</code> ; dashboard có cảnh báo DLQ.
TC-09	Load test	Gửi 500 log trong 2 giây và kích bản tải lớn hơn bằng k6.	<code>test.sh</code> xác nhận 500/500 persisted và 500/500 classified trong 31.7 giây; k6 burst đạt 250 request/giây với p95 HTTP 8.67 ms nhưng sinh 501 iteration do boundary của constant-arrival-rate.
TC-10	Telegram	Link chat bằng token TTL, gửi lỗi nghiêm trọng.	Chat nhận notification hợp lệ; unlink hoặc unsubscribe ngừng gửi.

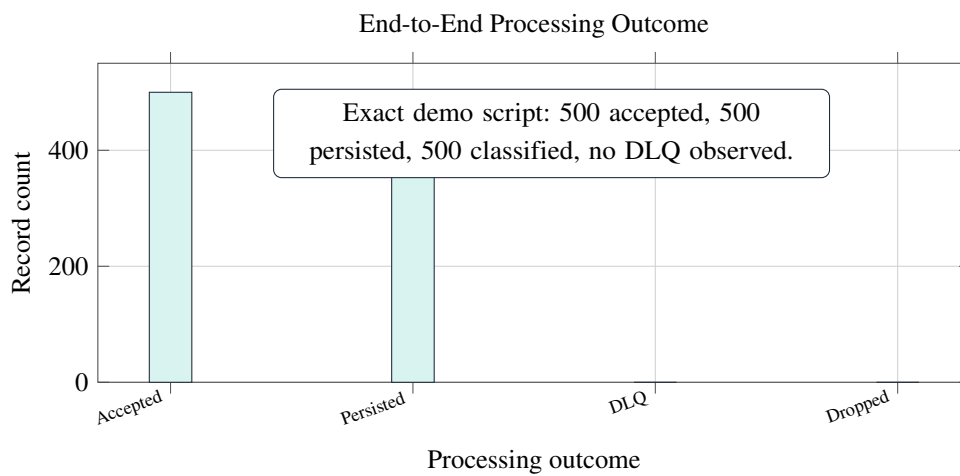
Các biểu đồ trong các hình tiếp theo dùng nhãn tiếng Anh và được điền bằng dữ liệu đo ngày 05/07/2026. Vì hệ thống chưa ghi timestamp từng stage trong record, một số cột là latency proxy ở mức batch/drain thay vì p95 per-message.



Hình 7: Khung đồ thị độ trễ pipeline theo từng stage



Hình 8: Khung đồ thị throughput ingest theo thời gian



Hình 9: Khung đồ thị kết quả xử lý end-to-end

9 Phương pháp đối chiếu với tài liệu mẫu

Các PDF báo cáo mẫu có một số đặc điểm chung đáng học: phần mở đầu nêu bối cảnh và động lực rõ ràng; phần tóm tắt liệt kê đóng góp kỹ thuật; mục lục, danh mục hình và danh mục bảng xuất hiện trước body; phần Nội dung và phương pháp không chỉ mô tả công nghệ mà đi vào yêu cầu, kiến trúc, bảng thành phần, dữ liệu và luồng tuần tự; phần Kết quả có môi trường triển khai, ma trận testcase và ảnh/đồ thị minh họa. Báo cáo này áp dụng các đặc điểm đó bằng cách tăng số bảng, tăng số sequence diagram, thêm catalog dữ liệu/topic/runtime state và chuyển các danh sách gạch đầu dòng thành bảng hoặc đoạn văn có tiêu đề phụ.

III. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

Chương này đánh giá hệ thống dựa trên những gì có thể chứng minh từ code, cấu hình và benchmark chạy thật. Các số liệu dưới đây được đo trên Docker Compose cục bộ ngày 05/07/2026; chúng phù hợp để đánh giá prototype nhưng chưa thay thế benchmark production nhiều node.

1 Môi trường triển khai

Hệ thống được triển khai bằng Docker Compose. Bảng 9 mô tả các service chính trong môi trường hiện tại. Môi trường benchmark dùng Linux 7.1.1, 20 logical CPU, RAM khoảng 15.4 GiB, Docker 29.5.3 và k6 2.0.0. Web server được publish ở port 3001 theo .env, ingest HTTP ở port 8085 và gRPC ingest ở port 50051.

Bảng 9: Các service chính trong môi trường Docker Compose

Service	Port/bề mặt	Vai trò trong demo
redpanda	Kafka API nội bộ, Pandaproxy nội bộ	Broker cho topic log, alert và DLQ.
benthos-unified	Internal consumer/producer	Tiêu thụ logs-ingested, route sang logs-normalized và alerts-ingested.
classifier	Internal worker	Tiêu thụ normalized log và produce enriched log.
benthos-persist	Internal consumer/HTTP client	Ghi ClickHouse theo batch và fallback DLQ.
clickhouse	HTTP/native theo Compose	Lưu log enriched, tags và materialized view.
redis	6379 nội bộ/host nếu publish	Pub/sub event, dedup state, Telegram queue và transient state.
postgres	PostgreSQL theo Compose	Lưu user/RBAC.
server	Web/API theo Compose	Phục vụ dashboard, alerts, metrics, config, REST API và WebSocket.
alert-worker	Internal worker	Deduplicate lỗi và sinh alert/Telegram job.
telegram	Telegram Bot API outbound	Link chat và gửi notification.

2 Kết quả hiện thực theo mã nguồn

Bảng 10 tổng hợp mức độ hoàn thiện của các nhóm chức năng. Cột đánh giá phân biệt phần đã chạy qua benchmark, phần có implementation nhưng còn lệch semantics, và phần cần hardening thêm.

Bảng 10: Mức độ hoàn thiện chức năng theo mã nguồn

Nhóm chức năng	Trạng thái	Nhận xét
Ingest qua broker	Đã có	Ingest worker HTTP/gRPC publish vào logs-ingested; benchmark smoke và burst xác nhận HTTP 202 và dữ liệu vào pipeline.

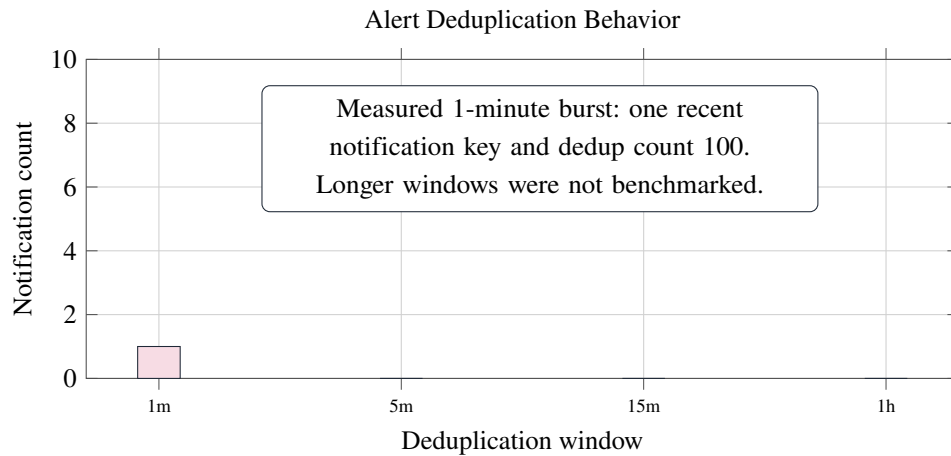
Nhóm chức năng	Trạng thái	Nhận xét
Chuẩn hóa payload	Đã có	Benthos unified xử lý value wrapper và sinh Trace_ID.
Classification	Đã đo path, chưa đo quality	Worker Python tạo 100/100 tag trong 7.203 giây; chưa có precision/recall/F1 trên dataset có nhãn.
Persistence	Đã có	Pipeline persist ghi ClickHouse bằng batch và có fallback DLQ.
Metrics	Đã có nền tảng và API đo được	Có materialized view theo giờ và metrics page; API analytics chạy 50 req/s với p95 5.16 ms khi dùng session token có sẵn.
Alert deduplication	Có nhưng semantics lệch yêu cầu exactly-one	Redis/Lua state có thật; 100 lỗi giống nhau tạo count 100 và một recent notification key, nhưng alert stream vẫn phát 100 update event.
Telegram integration	Đã có	Bot Go có link/subscription/status và tiêu thụ outbound queue; cần test token thật.
Historical RBAC	Có cơ sở	User/RBAC được lưu trong PostgreSQL; API lịch sử đọc session token và allowed apps.
Live RBAC	Đã có lọc theo subscription app	Admin nhận global stream; engineer subscribe theo allowed_apps. Cần tiếp tục test negative case bằng nhiều application.
Configuration management	Cần chuẩn hóa	Topic, port, credential, image tag và demo account còn phân tán.

3 Đánh giá luồng xử lý

Luồng xử lý chính đã thể hiện được mô hình pipeline hợp lý. Producer không cần biết ClickHouse, classifier hoặc alert worker. Unified pipeline thực hiện normalization và routing. Classifier enrichment tách khỏi persistence. Alert path tách khỏi log path để ERROR/CRITICAL có thể được xử lý riêng. Thiết kế này phù hợp với hệ thống cần tăng trưởng theo từng nhánh consumer [10, 11].

Điểm cần thận trọng là sự phụ thuộc vào Redis cho live event và alert state. Khi Redis bị flush hoặc restart, dashboard có thể mất trạng thái transient, alert recent state và token Telegram chưa sử dụng. Điều này không sai với một runtime store, nhưng cần phân biệt với dữ liệu cần audit dài hạn. Nếu alert history cần truy vết, cần persist sang ClickHouse/PostgreSQL hoặc một bảng audit riêng thay vì chỉ giữ trong Redis.

Hình 10 là khung đồ thị dùng để kiểm tra semantics alert sau khi chạy testcase. Tất cả nhãn đồ thị dùng tiếng Anh theo yêu cầu.



Hình 10: Khung đồ thị đánh giá hành vi deduplication của alert

4 Đánh giá bảo mật và phân quyền

Bảng 11 trình bày các rủi ro kỹ thuật quan trọng. Ma trận này ưu tiên các rủi ro có ảnh hưởng tới dữ liệu log và vận hành, vì log có thể chứa thông tin nhạy cảm và metadata nội bộ. Thiết kế security/correctness nên ưu tiên fail-closed cho stream và API nhạy cảm [12, 5].

Bảng 11: Ma trận rủi ro kỹ thuật và hướng giảm thiểu

Rủi ro	Tác động	Khả năng	Hướng giảm thiểu
Producer ngoài gọi trực tiếp broker	Cao	Trung bình	Xây dựng ingestion API riêng có API key, schema validation, rate limit và audit log; giữ Pandaproxy trong network nội bộ.
Non-admin nhận log ngoài phạm vi	Cao	Cao	Lọc WebSocket event server-side theo <code>allowed_apps</code> ; không subscribe global log stream cho user thường.
Alert noise do lỗi lặp lại	Trung bình	Cao	Định nghĩa dedup window và threshold rõ; test số notification trên cùng message hash.
Mất trạng thái Redis transient	Trung bình	Trung bình	Phân loại dữ liệu transient/audit; persist alert history quan trọng; có runbook khi Redis flush.
ClickHouse insert lỗi kéo dài	Cao	Trung bình	Theo dõi DLQ, cảnh báo insert error rate, replay tool và backpressure policy.
Credential demo bị dùng trong môi trường thật	Cao	Trung bình	Tách secret bằng biến môi trường, secret store hoặc file không commit; fail startup nếu dùng default trong production.
Classifier gán nhãn sai	Thấp đến trung bình	Chưa rõ	Đánh giá bằng dataset có nhãn, công bố precision/recall/F1 và cho phép fallback không tag khi model lỗi.

5 Đánh giá dữ liệu và truy vấn

ClickHouse schema hiện phù hợp với demo log analytics: native level enum, timestamp độ chính xác millisecond, UUID trace và tags array. Materialized view theo giờ giúp metrics page không phải scan mọi log mỗi lần. Tuy nhiên, ordering key (Timestamp, Trace_ID) cần được đánh giá bằng query workload. Bảng 12 là khung workload cần chạy để quyết định tối ưu schema.

Bảng 12: Khung workload truy vấn cần đo trên ClickHouse

Truy vấn	Mục đích	Chỉ số cần ghi
Recent logs by time	Nạp dashboard gần đây	latency p50/p95, rows scanned, bytes read.
Errors by application	Alerts/metrics theo service	latency, hiệu quả data skipping theo application/level.
Trace lookup	Điều tra một request	latency lookup theo Trace_ID.
Tag filter	Xem nhóm lỗi theo classifier	latency với has (Tags, ...) và tỉ lệ scan.
Hourly health	Chart error rate	latency đọc materialized view, độ trễ cập nhật sau insert.
DLQ replay check	Kiểm tra dữ liệu mất/khôi phục	số bản ghi replay thành công và lỗi lặp lại.

6 Kịch bản thử nghiệm hiện có trong mã nguồn

Các script trong logrider/scripts cung cấp khung demo nhưng chưa đủ để kết luận sản phẩm. Bảng 13 mô tả từng script và điểm cần sửa trước khi dùng làm bằng chứng nghiệm thu.

Bảng 13: Script kiểm thử và vận hành hiện có

Script	Mục đích	Lưu ý
setup-topics.sh	Tạo topic Redpanda	Có tạo logs-ingested lặp; cần chuẩn hóa partition và danh sách topic.
test-simple.sh	Gửi 5 log tương ứng 5 level	Gọi localhost:8082; cần chạy đúng network hoặc publish Pandaproxy.
test.sh	Gửi 500 log trong 2 giây bằng k6	Cần ghi kết quả persisted, latency và resource usage; phụ thuộc host access tới Pandaproxy.
test-alert.sh	Gửi nhiều log CRITICAL	Cần assert số alert/notification theo semantics dedup cụ thể.
test-extreme.sh	Kịch bản tải lớn	Cần định nghĩa lại vì comment và biến có thể chưa khớp với số log thực gửi.
verify_features.sh	Kiểm tra login, API, metrics, worker output	Cần rà soát channel Redis trong script so với channel thực tế ws-events/alerts-stream.
cleanup.sh	Xóa dữ liệu ClickHouse và Redis	Hữu ích để reset demo; cần cảnh báo rõ không dùng nhầm ở môi trường có dữ liệu thật.

7 Bảng kết quả thực nghiệm

Bảng 14 tổng hợp các benchmark đã chạy cho phiên bản hiện tại. Với các kịch bản không có latency per-message, cột latency ghi p95 HTTP nếu có, hoặc thời gian drain/batch đo được.

Bảng 14: Bảng kết quả benchmark đã đo

Kịch bản	Thông lượng	Latency p95	Ghi chú
5 log smoke test	2.71 persisted logs/s	1844 ms drain	5/5 persisted; 5/5 tags xuất hiện sau đó.
500 logs / 2 seconds	250 req/s ingest target; 15.75 persisted logs/s	8.67 ms HTTP p95; 31.7 s drain	test.sh: 500/500 persisted và 500/500 classified.
Alert burst 100 same errors	15.83 persisted logs/s	6.317 s drain	100/100 persisted; 100 alert stream update; một recent notification key; dedup count 100; Telegram queue 0 khi chưa subscribe.
ClickHouse degraded	20 accepted while storage down	Không đo p95; 12 s observation	20 accepted; 7 records vào dlq-clickhouse; 13 records persisted sau khi ClickHouse hồi phục.
WebSocket live dashboard	47.09 sessions/s	1.28 ms connect p95	1460/1460 WebSocket upgrade checks pass bằng session token có sẵn.
Classifier batch	13.88 tagged logs/s	7203 ms batch drain	100/100 records persisted và 100/100 tags trong log_tags.

8 Đánh giá tổng hợp

Ở trạng thái hiện tại, LogRider có đủ các thành phần để chứng minh một pipeline log monitoring end-to-end ở mức prototype. Điểm mạnh là kiến trúc được tách theo trách nhiệm, có message broker, có storage phân tích, có runtime state, có alert worker và có UI. Điểm yếu là các phần vận hành quan trọng chưa được đóng lại bằng số liệu và testcase: RBAC live, alert semantics, DLQ replay, latency end-to-end, classifier quality và cấu hình production. Vì vậy, kết luận hợp lý là hệ thống đã có nền tảng kỹ thuật nhưng cần một vòng hardening trước khi được xem là production-ready.

IV. KẾT LUẬN

Chương kết luận tổng hợp đóng góp, hạn chế và hướng phát triển. Nội dung được viết theo hướng cân bằng: ghi nhận những phần pipeline đã có cơ sở thực thi, đồng thời chỉ ra các hạng mục cần hoàn thiện trước khi xem hệ thống như một nền tảng monitoring production-grade.

1 Tổng kết

Báo cáo đã phân tích LogRider như một hệ thống log monitoring theo kiến trúc event-driven. Dựa trên mã nguồn hiện tại, hệ thống đã triển khai được pipeline chính gồm ingest qua Redpanda/Pandaproxy, normalize bằng Benthos, classify bằng worker Python, persist vào ClickHouse, alert bằng Redis-backed worker, WebSocket dashboard bằng Bun server và Telegram notification bằng Go bot. Thiết kế này thể hiện đầy đủ các tầng quan trọng của một mini-project phần mềm: data ingestion, stream processing, storage, runtime state, serving API, realtime UI và integration notification.

Phần Nội dung và phương pháp đã được mở rộng theo hướng phân tích–thiết kế hệ thống: yêu cầu chức năng, yêu cầu phi chức năng, kiến trúc tổng quan, sequence diagram, schema dữ liệu, topic catalog, Redis state, RBAC matrix, test matrix và benchmark cục bộ. Các bảng và đồ thị trong chương kết quả đã được điền bằng số liệu chạy thật thay vì số liệu giả.

2 Kết quả đạt được

Kết quả đạt được ở mức mã nguồn gồm schema ClickHouse rõ ràng, pipeline Benthos có fan-out và fallback, worker classifier có batch processing, alert worker có Redis Lua state, web server có dashboard/API/WebSocket, PostgreSQL có user/RBAC cơ bản và Telegram bot có luồng link/subscription/outbound. Các kết quả này cho thấy LogRider không chỉ là một trang dashboard tĩnh mà là một pipeline nhiều thành phần có luồng dữ liệu thực sự.

Kết quả chưa thể kết luận gồm năng lực production throughput, latency từng stage ở mức per-message, độ chính xác classifier, hành vi khi Redis lỗi, độ đúng của alert deduplication nếu requirement cuối cùng là exactly-one notification và negative test RBAC với nhiều application. Những phần này cần instrumentation và testcase sâu hơn.

3 Hạn chế

Hạn chế đầu tiên là benchmark mới ở phạm vi một máy Docker Compose, chưa phải production benchmark. Hạn chế thứ hai là alert deduplication cần thống nhất semantics trước khi viết test nghiệm thu nếu yêu cầu là một notification duy nhất trong một phút. Hạn chế thứ ba là ClickHouse schema chưa được tối ưu dựa trên query workload lớn. Hạn chế thứ tư là cấu hình vận hành còn phân tán, bao gồm topic name, credential mặc định, image tag và demo user. Hạn chế cuối cùng là classifier mới đo throughput path, chưa đo độ chính xác nhân.

4 Hướng phát triển

Hướng phát triển quan trọng nhất là xây dựng ingestion API riêng đứng trước broker. API này cần xác thực producer bằng API key hoặc token, validate schema, rate limit theo application, ghi audit log và publish vào logs-ingested. Khi đó Pandaproxy được giữ trong network nội bộ thay vì trở thành public ingress.

Hướng phát triển tiếp theo là sửa RBAC live stream. Web server cần xác định allowed applications ngay khi mở WebSocket và lọc từng event trước khi gửi. Nếu một event không có application scope rõ ràng, hệ thống nên

fail-closed với non-admin thay vì coi đó là global log. Đây là thay đổi ưu tiên cao vì ảnh hưởng trực tiếp tới bảo mật dữ liệu log.

Hướng thứ ba là chuẩn hóa alert semantics. Nếu sản phẩm cần một notification duy nhất trong một cửa sổ thời gian, alert worker cần window key, TTL và idempotency logic phù hợp. Nếu sản phẩm cần reminder tại các threshold, tài liệu phải ghi rõ số mốc và testcase phải assert số notification tương ứng. Cả hai thiết kế đều có thể hợp lý, nhưng không thể dùng một implementation để thỏa mãn hai semantics khác nhau.

Hướng thứ tư là mở rộng benchmark và observability nội bộ. Hệ thống đã có số liệu cục bộ cho ingest, API query, WebSocket connect, classifier batch, alert burst và ClickHouse degraded path; bước tiếp theo là đo latency từng stage bằng timestamp/correlation id, broker lag, ClickHouse insert error rate, Redis pub/sub throughput, Telegram delivery result và benchmark nhiều node. Khi hệ thống có số liệu chi tiết hơn, mới nên tối ưu partition, batch size, ClickHouse ordering key hoặc worker concurrency. Nguyên tắc này phù hợp với phương pháp đo hiệu năng theo workload thực tế thay vì tối ưu dựa trên giả định [13, 18].

Hướng cuối cùng là hardening triển khai. Dự án nên tập trung hóa cấu hình vào environment variables có tài liệu, bỏ credential demo khỏi môi trường thật, tránh image tag không cố định, bổ sung migration/versioning cho database, thêm runbook backup/restore, thêm dashboard DLQ và thiết lập CI kiểm thử pipeline bằng dữ liệu mẫu. Với các consumer hoặc sender có retry, cần dùng exponential backoff kèm jitter để tránh retry đồng bộ gây thêm tải khi dependency vừa phục hồi [16].

TÀI LIỆU THAM KHẢO

- [1] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [2] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [3] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: The Definitive Guide*. O'Reilly Media, 2017.
- [4] Jay Kreps. “The Log: What every software engineer should know about real-time data’s unifying abstraction.” LinkedIn Engineering Blog, 2013.
- [5] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [6] Charity Majors, Liz Fong-Jones, and George Miranda. *Observability Engineering*. O'Reilly Media, 2022.
- [7] James Turnbull. *The Art of Monitoring*. Turnbull Press, 2014.
- [8] Anton Chuvakin, Kevin Schmidt, and Chris Phillips. *Logging and Log Management*. Syngress, 2012.
- [9] Alex Petrov. *Database Internals: A Deep Dive into How Distributed Data Systems Work*. O'Reilly Media, 2019.
- [10] Sam Newman. *Building Microservices*. O'Reilly Media, 2nd edition, 2021.
- [11] Chris Richardson. *Microservices Patterns*. Manning, 2018.
- [12] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.
- [13] Brendan Gregg. “The USE Method.” Brendan Gregg’s Blog, 2012.
- [14] Tyler Treat. “You Cannot Have Exactly-Once Delivery.” Brave New Geek, 2015.
- [15] Martin Fowler. “Event Sourcing.” martinowler.com, 2005.
- [16] Marc Brooker. “Exponential Backoff and Jitter.” AWS Architecture Blog, 2015.
- [17] Unmesh Joshi. *Patterns of Distributed Systems*. martinowler.com catalog.
- [18] Luiz André Barroso, Jeffrey Dean, and Urs Hölzle. “The Tail at Scale.” Communications of the ACM, 2013.
- [19] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018.